Руководство пользователя DITA Studio

Содержание

- 1 О DITA Studio
  - 1.1 Как устроен режим «Автор»
  - 1.2 Сохранность правок
- 2 Основы DITA
  - 2.1 Типы топиков
  - 2.2 Карты публикации
  - 2.3 Ключи и ключевые слова
- 3 Работа с проектом
  - 3.1 Создание проекта с нуля
  - 3.2 Открытие и создание проекта
  - 3.3 Редактирование текста и структуры
  - 3.4 Вставка изображений, ссылок и сносок
  - 3.5 Таблицы в режиме «Автор»
  - 3.6 Проверка проекта и поиск
  - 3.7 Условия сборки и публикация
    - 3.7.3 Оформление DOCX
  - 3.8 Переименование и перенос без битых ссылок
  - 3.9 Рецензирование и сравнение версий
  - 3.10 Перевод топика через XLIFF
- 4 Расширение редактора
  - 4.1 Расширение редактора
- 5 Справочные материалы
  - 5.1 Главное окно
  - 5.2 Сочетания клавиш
  - 5.3 Структура топиков
  - 5.4 Что редактируется только в исходном коде
- 6 Устранение неполадок
  - 6.1 PDF не экспортируется
  - 6.2 Вкладка «Предпросмотр» пустая
- 7 Глоссарий

- 7.1 топик
- 7.2 карта
- 7.3 ключ

1 О DITA Studio

DITA Studio — редактор технической документации в формате DITA 1.3 с наглядным режимом «Автор» и проверкой по встроенному словарю элементов.

Проект в DITA Studio — это папка на диске: одна или несколько карт (`.ditamap`) и топики рядом с ними. Открыть проект можно через Файл → Открыть папку проекта….

Каждый открытый документ показывается в трёх синхронных режимах — переключаться между ними можно вкладками над областью редактирования:

- **Автор** — структурированный WYSIWYG: каждый блочный элемент редактируется отдельно, а вставка и правка проверяются по модели содержимого DITA, поэтому получить невалидный документ через этот режим невозможно.
- **Исходный код** — обычный XML-редактор с подсветкой синтаксиса и автодополнением тегов.
- **Предпросмотр** — то, как топик будет выглядеть после публикации: как страница HTML, как настоящий PDF с разбивкой на страницы, как «живые» листы заданного размера, которые пересчитываются при каждом нажатии клавиши (в том числе рядом с текстом), или приближённо как документ Word.[1]

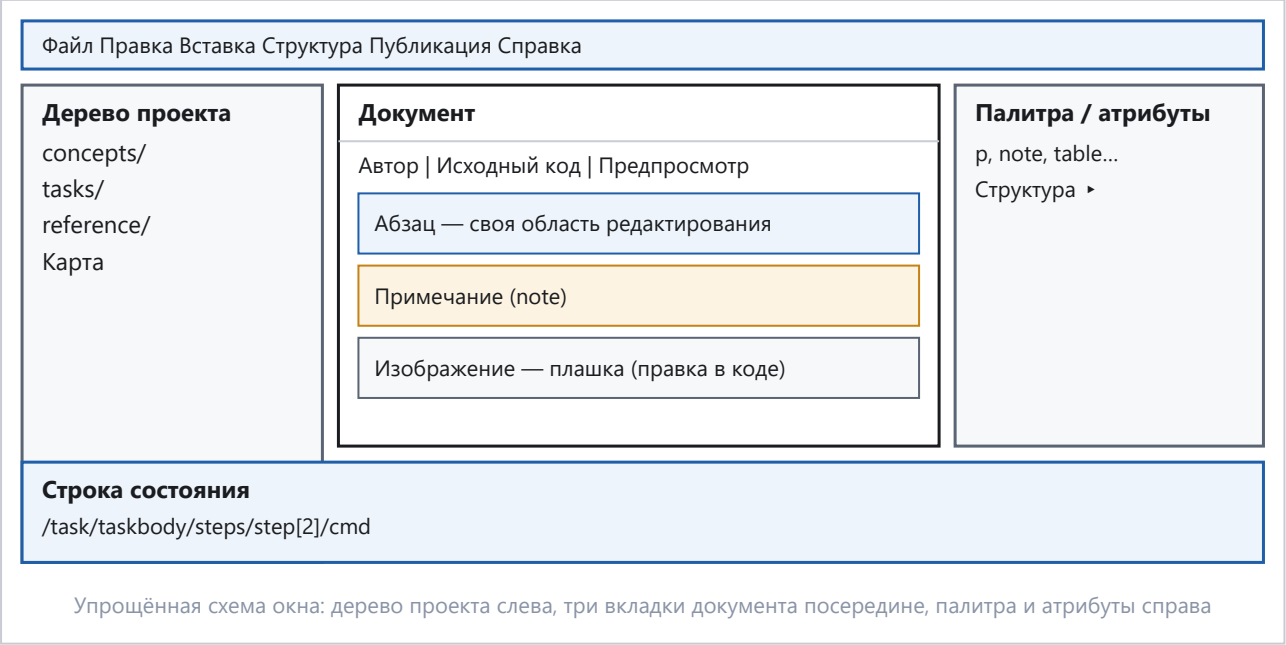
Кроме редактирования, в DITA Studio есть инструменты для сопровождения документации: рефакторинг ссылок, рецензирование правок и сравнение версий, перевод через XLIFF, подключение собственных специализаций и плагинов — см. разделы «Работа с проектом» и «Расширение редактора».

Отмена (**Ctrl+Alt+Z**) относится к структурным правкам и хранится как снимки XML; обычный набор текста отменяется средствами самого текстового поля по **Ctrl+Z** — это два независимых стека отмены.

> **Совет:** Если не помните, что означает элемент или какие атрибуты у него есть, поставьте курсор внутрь него и нажмите **F1** — появится справка из встроенного словаря: описание, допустимые дочерние элементы и атрибуты.

Под меню — панель инструментов: оформление текста (полужирный, курсив, подчёркнутый, код), вставка ссылки, изображения, сноски, раздела, абзаца, списков и таблицы, операции со строками и столбцами таблицы, перемещение и удаление элемента, проверка и сборка. Название кнопки показывается при наведении указателя.

Рисунок 1. Основные области главного окна



Упрощённая схема окна: дерево проекта слева, три вкладки документа посередине, палитра и атрибуты справа

Сноски

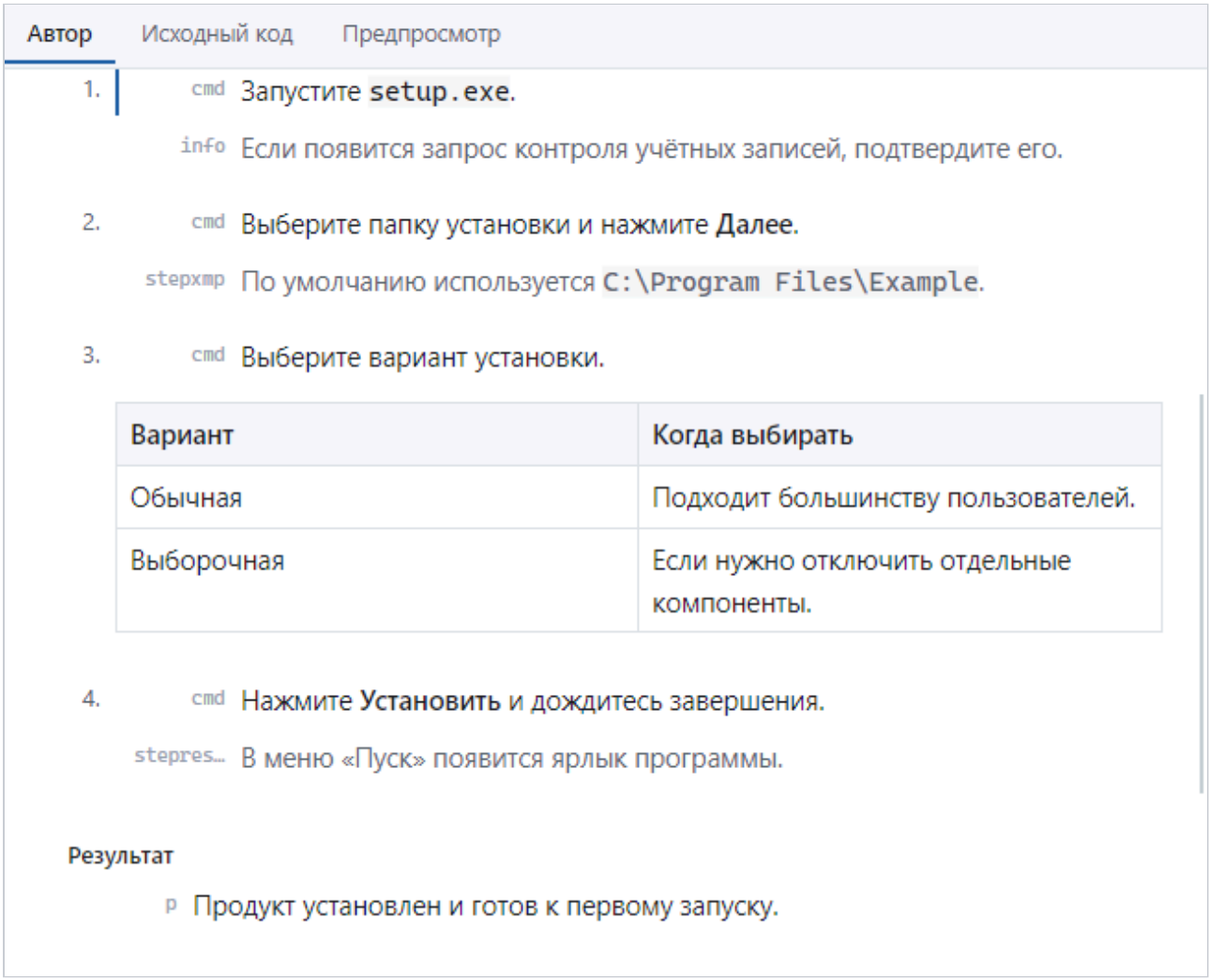
1. Страницы показывает встроенный в редактор браузер (Chromium) — он же печатает PDF со своими колонтитулами, см. задачу «Условия сборки и публикация». ↩

1.1 Как устроен режим «Автор»

Документ превращается в набор редактируемых блоков, а не в один сплошной текст — это и не даёт получить невалидную структуру.

Каждый блочный элемент (абзац, шаг, примечание, ячейка таблицы) — это собственная область редактирования. Фразовые элементы, у которых есть прямой текст (`b`, `i`, `codeph`, `uicontrol` и подобные), показываются как оформленный текст прямо внутри абзаца. Сноски (`fn`) не разрывают текст: в строке — плашка «сноска 1», а текст сноски правится в области **Сноски** внизу топика. Блоки-обёртки — `div`, `bodydiv`, `section`, `sectiondiv`, `fig` — обведены полосой слева с подписью, чтобы было видно, где они начинаются и кончаются.

Рисунок 2. Задача в режиме «Автор»



Enter в конце блока открывает подсказку — какой элемент вставить дальше, с описанием выбранного справа. Первая строка — обычное действие Enter (следующий абзац, шаг, пункт списка), поэтому двойной Enter работает как раньше; набор текста фильтрует список, Esc закрывает. Выключается пунктом Структура → Подсказка по Enter.

Окно подсказки можно растянуть за любой из четырёх углов — выбранный размер запоминается. Щелчок по свободному месту снимает выделение блока и текста, а полоса прокрутки следует за курсором, в том числе внутри длинного абзаца.

Чтобы выйти из таблицы, `div` или пункта списка, нажмите Ctrl+Enter — откроется то же меню допустимых элементов, но для места после текущего блока и после внешних блоков (в ячейке — после самой таблицы). То же меню открывает двойной щелчок по свободному месту — под последним блоком, между блоками или в пустом контейнере; двойной щелчок в тексте по-прежнему выделяет слово.

Клавиши Enter, Backspace и Tab не просто редактируют текст — они выполняют структурные операции (разбить абзац, объединить с предыдущим, превратить шаг в подшаг) и перед применением проверяются по конечному автомату модели содержимого. Если операция сделает документ невалидным, редактор её не выполнит.

Выравнивание текста — по левому краю, по центру, по правому краю, по ширине — задаётся кнопками на панели инструментов или в контекстном меню Оформление → Выравнивание и действует на абзац, заголовок, пункт списка, термин, определение, команду шага или ячейку под курсором. Знать CSS не нужно: в файл записывается класс `outputclass` (у ячейки таблицы — атрибут `align`), и публикация в HTML, PDF и DOCX выравнивает текст по нему.

Размер шрифта выбирается в списке **Размер** на панели или в меню Оформление → Размер шрифта так же, как в Word: выделенный текст получает выбранный размер, а без выделения у курсора появляется заготовка — дальше текст печатается уже этим размером. Пункт **Обычный** возвращает размер по умолчанию. Цвет текста — кнопка **A** с палитрой на панели или Оформление → Цвет текста; работает так же, **Без цвета** снимает цвет.

Выделение текста цветом — маркер, как «Цвет выделения текста» в Word: кнопка **A** на жёлтом фоне на панели или Оформление → Маркер. В палитре красный, жёлтый, зелёный, синий, **Другой цвет…** и **Нет цвета**. Выделенный текст закрашивается сразу; если ничего не выделено, включается режим кисти — закрашивается всё, что вы выделите мышью, пока не нажата клавиша **Esc**. В DOCX основные цвета становятся настоящим выделением текста Word.

Если изменить заголовок топика и перейти в другой блок, редактор спросит, переименовать ли файл по новому заголовку. При согласии файл получает имя по заголовку (как при создании документа), а все ссылки на него в проекте обновляются; при отказе ничего не меняется.

Оформление можно задать сразу целому контейнеру — `note`, `section`, `div`: щелкните по его рамке (полоса слева или подпись), затем выберите выравнивание, размер или цвет. Класс запишется на сам контейнер и подействует на весь текст внутри; собственное оформление абзаца внутри главнее.

Блоки можно выделять целиком не только по одному: нажмите в тексте блока и потяните к другому блоку — обведутся все блоки между ними (продолжить можно **Shift**+щелчком). **Delete** удаляет выделенное, **Esc** снимает выделение, а **Enter** или пункт Структура → Обернуть в… помещает выделенные блоки в другой элемент — примечание, раздел, `div`; в окне будут только допустимые варианты.

Ячейки таблицы выделяются мышью: нажмите в одной ячейке и потяните к другой — подсветится прямоугольник (продолжить можно **Shift**+щелчком). Правая кнопка мыши на выделении или значки на панели вставляют и удаляют строки и столбцы по числу выделенных, объединяют ячейки в одну, очищают, выравнивают. Значок **Границы** работает как в Word: нижняя, верхняя, левая и правая граница, внешние и внутренние, внутренние линии; край всей таблицы задаётся для всей его длины сразу.

Изображение из пункта Вставка → Изображение вставляется рисунком: блок `fig` с названием и изображением внутри. Слева от названия видна подпись «Рисунок N.» — при публикации она печатается по центру, а в DOCX номер — поле Word. Изображение, уже стоящее в абзаце, оформляется командой **Оформить изображение как рисунок**. Формат подписи — «Рисунок 1. Название» или «Рисунок 1 — Название» — выбирается в настройках оформления DOCX.

Изображения показываются самой картинкой — и отдельным блоком, и внутри строки текста. Чтобы изменить размер, наведите указатель на картинку и потяните за квадратный маркер в правом нижнем углу: пропорции сохраняются, а в файл записывается атрибут `width`, который учитывает публикация.

Щелчок по изображению открывает окно «Свойства изображения»: подпись (`alt`), ширина и файл; щелчок по плашке ссылки без текста — окно «Свойства ссылки»: текст, адрес и область (`scope`). **Enter** или щелчок мимо окна записывает введённое, **Esc** отменяет; запись отменяется одним шагом.

Ячейки таблицы копируются и вставляются как в Excel: выделите прямоугольник ячеек мышью, **Ctrl+C** (или **Ctrl+X**) и **Ctrl+V** в другой ячейке. Таблица при необходимости удлиняется; текст с табуляциями из Excel или Word вставляется так же, а вне таблицы становится новой таблицей. Чтобы снять нумерацию абзаца, поставьте курсор в его начало и нажмите **Backspace**. Команда «Изображение» спрашивает, вставлять ли рисунок с нумерацией или обычное изображение у курсора.

Кнопки «Ж», «К», «Ч» и «</>» на панели показывают оформление текста под курсором и снимают его повторным нажатием. Пометка `status` над блоком цветная (new — зелёная, changed — оранжевая, deleted — красная), а панель «Правки» справа собирает такие блоки по всему проекту и переводит к ним щелчком. Правая кнопка по таблице или рисунку — «Наименование таблицы…» / «Наименование рисунка…». Размер картинки меняется мышью в любом блоке, в том числе в ячейке таблицы.

Справа — три панели о текущем элементе: **Атрибуты** (имя элемента, атрибуты с описанием из каталога, «Добавить атрибут»; у перечислений — список значений, у `product` — флажки продуктов проекта), **Вставка** (палитра допустимых элементов) и **Структура** (дерево топика, переход щелчком).

Рисунок 3. Панель «Атрибуты» для абзаца

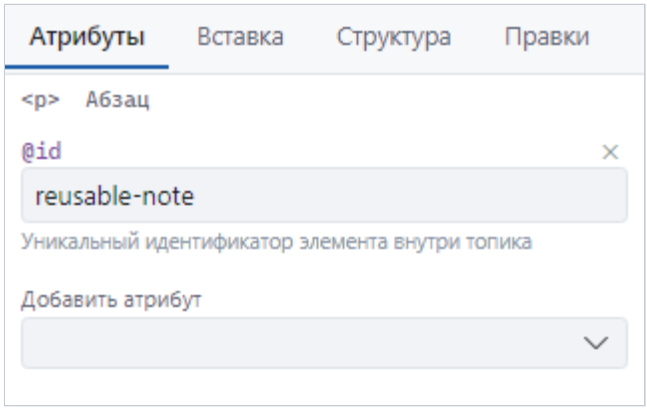
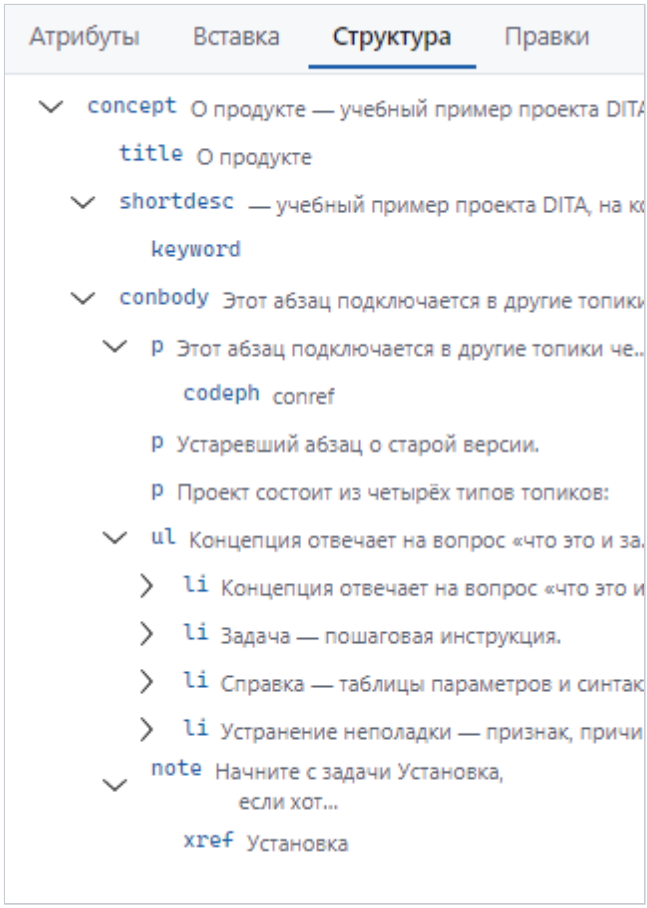


Рисунок 4. Панель «Структура»



Важно: Не все элементы можно редактировать в режиме «Автор». Инлайн-элементы без собственного текста — вставки по `conref`/`conkeyref` — показываются нередактируемой «плашкой». Точную правку такого элемента нужно делать в режиме **Исходный код**; полный список — в справочном топике Что редактируется только в исходном коде.

1.2 Сохранность правок

Как редактор защищает документы от сбоя при записи, от аварийного завершения и от изменений, сделанных другими программами.

1.2.1 Надёжное сохранение

Файл сначала записывается во временный файл в той же папке, а затем заменяет оригинал. Если во время сохранения пропадёт питание или закончится место на диске, на диске останется прежняя версия документа, а не обрезанная новая.

1.2.2 Восстановление после сбоя

Каждые 30 секунд редактор копирует текст вкладок с несохранёнными правками в папку данных программы: `%LOCALAPPDATA%\DitaStudio\Recovery` в Windows, `~/.local/share/DitaStudio/Recovery` в Linux, `~/Library/Application`

`Support/DitaStudio/Recovery` в macOS. Файлы проекта при этом не меняются. Копия удаляется, когда документ сохранён или вы отказались от правок при закрытии.

Если редактор завершился аварийно, при следующем открытии того же проекта появится окно Восстановление после сбоя со списком документов:

- **Да** — документы откроются с восстановленными правками. Правки ещё не сохранены: проверьте их и сохраните документ.
- **Нет** — копии удаляются.
- **Отмена** — решить позже. Копии будут предложены снова при следующем открытии проекта.

> **Примечание:** Пометка «файл с тех пор менялся» значит, что оригинал изменили после того, как была снята копия. Сравните версии, прежде чем сохранять.

1.2.3 Изменения другими программами

Редактор следит за файлами открытых вкладок. Если файл изменила другая программа, например система контроля версий или другой редактор:

- вкладка без несохранённых правок перечитывается автоматически;
- если во вкладке есть правки, редактор спросит, какую версию оставить. Выбор **Да** загружает версию с диска, а свою можно вернуть командой Правка → Отменить структурное изменение. Выбор **Нет** оставляет вашу версию: при сохранении она заменит файл на диске;
- если файл удалили или переименовали, вкладка остаётся открытой и помечается как несохранённая. Сохраните её, чтобы записать файл заново.

Перед сменой проекта и перед выходом редактор предлагает сохранить изменения. Если сохранить не удалось, окно не закроется и проект не сменится.

---

2 Основы DITA

2.1 Типы топиков

Чем отличаются концепция, задача, справка, устранение неполадки, статья глоссария и универсальный топик и какой тип выбрать для своего текста.

В DITA текст делится на небольшие самостоятельные части — топики. Снаружи все топики устроены одинаково: заголовок `title`, краткое описание `shortdesc`, пролог с метаданными `prolog`, тело и связанные ссылки `related-links`. Тип топика определяет, что можно положить в тело. DITA Studio знает эти правила: панель «Вставка» показывает только

допустимые элементы, а проверка сообщает, если порядок нарушен. Сводные таблицы — в Структура топиков.

2.1.1 Концепция

Отвечает на вопросы «что это?» и «зачем?»: описание продукта, принципа работы, понятия. Пошаговых инструкций в концепции быть не должно — их место в задаче, на которую концепция может сослаться. Тело `conbody` принимает абзацы, списки, таблицы, рисунки, примечания, блоки кода, разделы `section` и примеры `example`.

```
<concept id="about-backup">
  <title>Резервное копирование</title>
  <shortdesc>Копия защищает данные от сбоя диска.</shortdesc>
  <conbody>
    <p>Программа сохраняет копию раз в сутки…</p>
  </conbody>
</concept>
```

2.1.2 Задача

Инструкция, которую читатель выполняет по шагам. Части тела `taskbody` необязательны, но идут строго по порядку: условия `prereq`, контекст `context`, шаги, результат `result`, что делать при неудаче `tasktroubleshooting`, примеры `example`, дальнейшие действия `postreq`.

Шаги бывают нумерованные `steps`, в любом порядке `steps-unordered` и свободным текстом `steps-informal`. Шаг `step` начинается с команды `cmd` — одного действия в повелительном наклонении. За ней могут идти пояснение `info`, подшаги `substeps`, пример `stepxmp`, варианты `choices` или `choicetable`, примечание; в конце — результат шага `stepresult`. Атрибут `importance` со значением `optional` помечает необязательный шаг.

В режиме «Автор» Enter в конце команды создаёт следующий шаг, Tab и Shift+Tab меняют уровень вложенности.

```
<taskbody>
  <prereq><p>Нужны права администратора.</p></prereq>
  <steps>
    <step><cmd>Откройте меню Сервис.</cmd></step>
    <step>
      <cmd>Выберите Резервная копия.</cmd>
      <stepresult><p>Откроется окно выбора папки.</p></stepresult>
    </step>
  </steps>
  <result><p>Копия появится в выбранной папке.</p></result>
</taskbody>
```

2.1.3 Справка

Сведения, которые не читают подряд, а ищут глазами: параметры, коды ошибок, команды, характеристики. Тело `refbody` состоит из разделов, синтаксиса `refsyn`, примеров, таблиц и таблиц свойств `properties`. Таблица свойств — это строки `property` из трёх ячеек: тип

`proptype`, значение `propvalue`, описание `propdesc`. В режиме «Автор» она правится по ячейкам.

Топик из одной таблицы удобно создавать заготовкой **Таблица (без заголовка)**: заголовок топика в ней пустой — при публикации он не печатается и в оглавление не попадает, а в карте и ссылках топик называется по подписи таблицы. Так же ведёт себя любой топик, у которого стёрт текст заголовка.

2.1.4 Устранение неполадки

Разбор проблемы по схеме «признак → причина → решение». Тело `troublebody` начинается с признака `condition`, за ним — один или несколько вариантов `troubleSolution`, каждый из причины `cause` и решения `remedy`. Решение содержит шаги, как в задаче, и при необходимости — кто их выполняет (`responsibleParty`). Располагайте варианты от самой вероятной причины к самой редкой.

2.1.5 Статья глоссария

Одна статья — один термин: вместо заголовка термин `glossterm`, вместо краткого описания — определение `glossdef`. Необязательное тело `glossBody` уточняет полную форму термина (`glossSurfaceForm`), сокращения и синонимы (`glossAlt` с `glossAcronym`, `glossAbbreviation`, `glossSynonym`), правила употребления.

Статью удобно подключать к тексту через ключ: `<abbreviated-form keyref="plc"/>` выводит сокращение из статьи, а если его нет — сам термин. Несколько коротких статей можно собрать в одном файле с корнем `glossgroup`.

2.1.6 Универсальный топик

Базовый тип `topic` с телом `body` без ограничений на структуру. Подходит для вводных страниц и текстов, которые не являются ни объяснением, ни инструкцией, ни справкой. Если текст ложится в специализированный тип, выбирайте его: структура помогает автору, читателю и проверке.

2.1.7 Учебные топики

Типы DITA Learning and Training (`learningOverview`, `learningContent`, `learningSummary`, `learningAssessment`, `learningPlan`) есть в каталоге редактора, но готовых заготовок для них нет. Создайте универсальный топик и в режиме «Исходный код» замените корневой элемент и строку `DOCTYPE`. Так же создаётся и группа глоссария `glossgroup`.

2.1.8 Вложенные топики

После тела топик может содержать дочерние топики. Это удобно для коротких связанных частей, которые не заслуживают отдельного файла, но в публикации получают свои заголовки.

2.2 Карты публикации

Карта задаёт, какие топики входят в публикацию, в каком порядке и с какой вложенностью. Обычная карта, карта книги и схема категорий решают разные задачи.

Карта не содержит текста — только ссылки на топики и сведения о публикации. Один и тот же топик может входить в несколько карт: например, в полное руководство и в краткую памятку.

2.2.1 Обычная карта

Элемент	Что делает
`title`	Заголовок публикации.
`topicref`	Ссылка на топик; вложенные ссылки — подразделы.
`topichead`	Заголовок раздела без собственной страницы; текст — в `topicmeta/navtitle`.
`topicgroup`	Группа ссылок без заголовка.
`keydef`	Определение ключа; в оглавление не попадает.
`mapref`	Подключение другой карты целиком — большая публикация собирается из частей.
`reltable`	Таблица соответствий: связи «Смотрите также» между топиками.

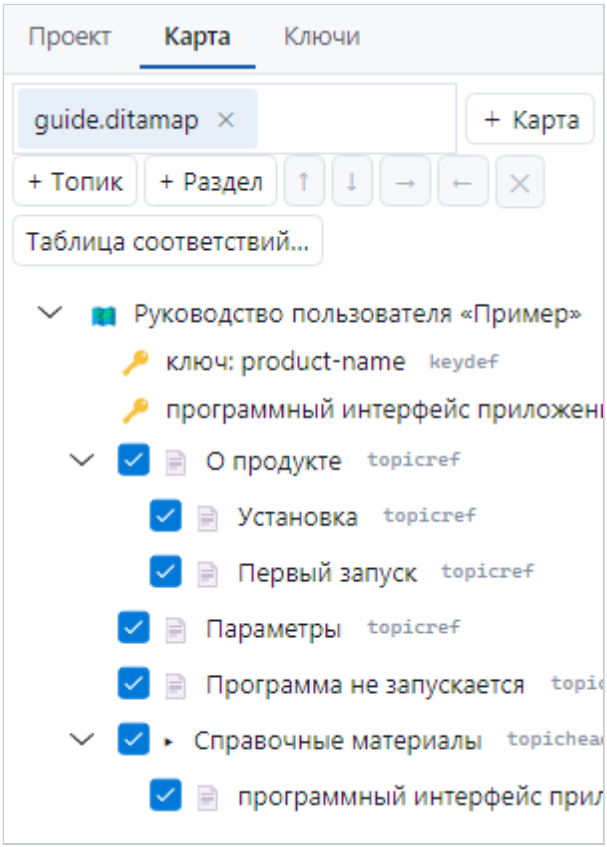
Полезные атрибуты ссылки на топик:

- `navtitle` (или элемент `topicmeta/navtitle`) — свой текст строки оглавления вместо заголовка топика;
- `processing-role` со значением `resource-only` — топик нужен для ссылок и ключей, но отдельной страницей в публикацию не выходит;
- `keyscope` — своя область ключей для ветки карты.

> **Примечание:** Условные атрибуты (`product`, `audience` и другие) действуют и на строки карты: `topicref` или `topichead` с исключённым значением уходит из публикации вместе со всей веткой — из оглавления, навигации и «Смотрите также».

В редакторе карта правится на вкладке **Карта** слева: кнопки добавления, перестановки и вложения строк, перетаскивание мышью (с проверкой по модели карты), флажок «публиковать» (`processing-role="resource-only"`), меню строки. Красные строки — битые ссылки; двойной щелчок по ним предлагает создать файл, выбрать другой или убрать строку.

Рисунок 5. Вкладка «Карта»



2.2.2 Карта книги

Для публикаций в виде книги (PDF, DOCX). Вместо `title` — заголовок `booktitle` с основным названием `mainbooktitle`, дальше:

- `frontmatter` — начало книги: перечни `booklists` (например, оглавление `toc`), предисловие, уведомления;
- `part` — части, внутри которых главы;
- `chapter` — главы, в них вкладываются обычные ссылки на топики;
- `appendix` — приложения;
- `backmatter` — завершающая часть.

Заготовка «Карта книги» уже содержит заголовок и оглавление.

2.2.3 Схема категорий

Карта особого вида: описывает допустимые значения условных атрибутов — например, что `product` может быть только `basic` или `pro`. DITA Studio знает структуру схемы и проверяет её, но значения атрибутов в топиках по ней не проверяет: список значений для условий сборки собирается из фактически использованных в проекте.

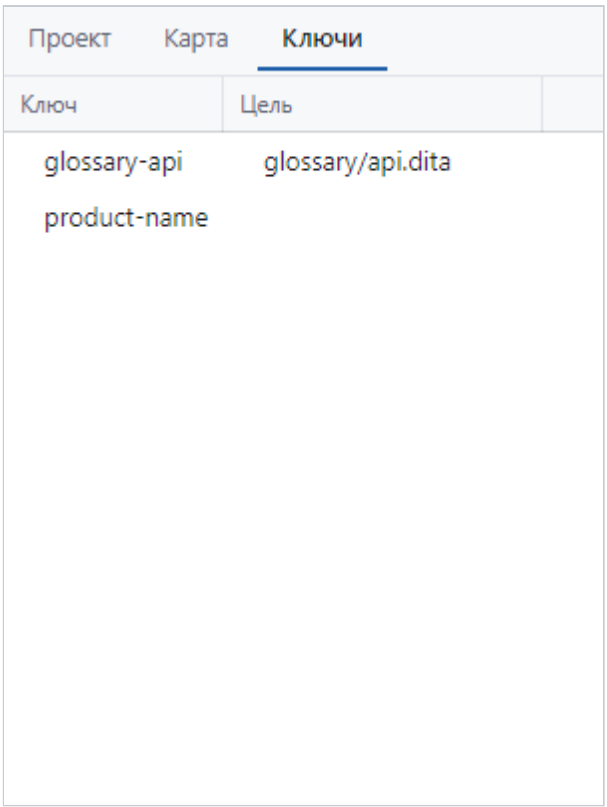
2.3 Ключи и ключевые слова

Ключ хранит значение — текст, ссылку, картинку или фрагмент — в одном месте карты, а топики ссылаются на него по имени. Там же: области ключей, ключевые слова в прологе и предметный указатель.

Топики ссылаются на имя ключа (`keyref="product"`), а значение задаётся один раз в карте. Поменялось название продукта или адрес — правится одна строка карты, а не каждый топик. Каждая карта может задать свои значения, поэтому из одних и тех же топиков собираются разные публикации. Например, название этого редактора — DITA Studio — в руководстве взято из ключа `product-name`.

Ключи корневой области всех карт проекта видны на вкладке **Ключи** слева: имя и цель (файл или адрес). Список пересобирается при сохранении карты.

Рисунок 6. Вкладка «Ключи»



2.3.1 Как определить и использовать ключ

Ключ определяет атрибут `keys` — обычно на `keydef` в карте. Одно определение может задавать несколько имён через пробел.

Что хранит	Определение в карте	Использование в топике
Текст	`keydef` с `topicmeta/keywords/keyword`	`<keyword keyref="product"/>`, `<ph keyref="…"/>`
Ссылку на топик	`<keydef keys="install" href="tasks/install.dita"/>` и `navtitle` для текста ссылки	`<xref keyref="install"/>`
Внешний адрес	`keydef` с `href="https://…"` и `scope="external"`	`<xref keyref="site"/>`
Картинку	`<keydef keys="logo" href="images/logo.png"/>`	`<image keyref="logo"/>`
Термин глоссария	`<keydef keys="plc" href="glossary/plc.dita"/>`	`<abbreviated-form keyref="plc"/>`
Общий фрагмент	`<keydef keys="shared" href="reusable/shared.dita"/>`	`<p conkeyref="shared/warning"/>` — вставляет элемент с `id="warning"`

Текст ключа берётся из `topicmeta/keywords/keyword`, а если его нет — из `topicmeta/navtitle`. В режиме «Автор» атрибут `keyref` ставится на вкладке «Атрибуты»: выделите элемент, выберите `keyref` в списке «Добавить атрибут» и впишите имя ключа.

2.3.2 Правила, о которых стоит помнить

- **Текст подставляется только в пустой элемент.** Так требует спецификация DITA: собственный текст элемента остаётся, и его можно использовать как запасное значение.
- **Первое определение побеждает.** Ключи собираются из всех карт проекта. Держите их в одном месте — например, в отдельной карте, подключённой через `mapref`.
- **Ключи пересобираются при сохранении.** Добавили `keydef` — сохраните карту, и ключ появится на вкладке «Ключи».
- **Необъявленный ключ — ошибка проверки.** F7 находит `keyref` и `conkeyref` на несуществующие ключи.

2.3.3 Области ключей

Атрибут `keyscope` на `topicref` или `topichead` создаёт для ветки карты свою область ключей. Ключ ищется сначала в своей области, потом в родительских, вплоть до корня. Так в одной публикации можно описать две модели устройства: в ветке `keyscope="basic"` ключ `model` означает «Базовая», в ветке `keyscope="pro"` — «Про», а топики используют один и тот же `keyref="model"`. К ключу конкретной области можно обратиться явно: `keyref="pro.model"`.

Общие ключи нескольких проектов можно хранить отдельно и подключать как проект-источник, см. Проекты-источники ключей.

2.3.4 Ключевые слова

Слово «ключевые» в DITA встречается в двух разных смыслах:

- `keyword` внутри определения ключа — это значение ключа, текст для подстановки;
- `keywords` в прологе топика (`prolog/metadata/keywords`) — это метаданные: слова, описывающие содержание топика для поиска и каталогизации. В текст публикации они не выводятся.

Элемент `keyword` в тексте без `keyref` просто помечает слово как ключевое понятие и выводится как обычный текст.

2.3.5 Предметный указатель

Термин указателя `indexterm` ставится прямо в текст топика или в ключевые слова пролога; вложенный `indexterm` — подпункт, а `index-see` и `index-see-also` дают отсылки «см.» и «см. также». Алфавитный указатель собирается при любой публикации: в конце единого HTML-файла (и PDF), отдельной страницей `index-terms.html` многостраничного сайта и полем «Указатель» в DOCX — номера страниц в нём Word проставит при обновлении полей.

```
<indexterm>резервная копия<indexterm>восстановление</indexterm></indexterm>
```

---

3 Работа с проектом

3.1 Создание проекта с нуля

Путь от пустой папки до готового HTML-сайта: структура папок, карта, первые топики, ключи, проверка и публикация.

3.1.1 Контекст

Проект DITA Studio — это обычная папка с картами (`.ditamap`), топиками (`.dita`), картинками и служебными файлами настроек `.ditastudio-*`. Отдельный «файл проекта» создавать не нужно.

Порядок действий

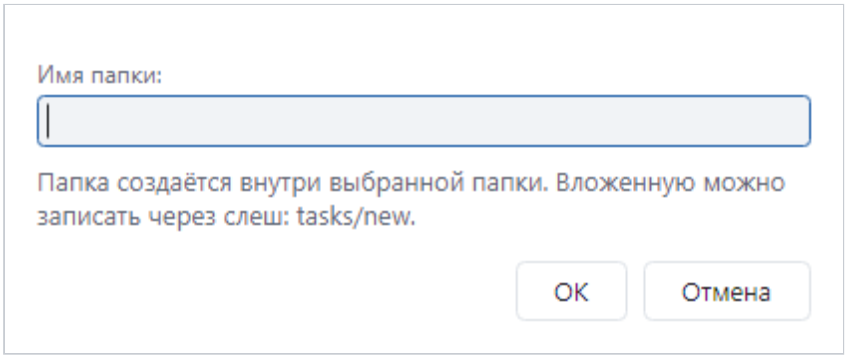
1. **Создайте в Проводнике папку проекта и подпапки для разных видов содержимого.**

   Диалог «Создать документ» предлагает только существующие папки, поэтому удобнее создать их заранее. Типичная структура:

```
MyProduct/
  myproduct.ditamap     главная карта публикации
  keys.ditamap          (по желанию) ключи отдельной картой
  concepts/             концепции
  tasks/                задачи
  reference/            справка
  troubleshooting/      устранение неполадок
  glossary/             статьи глоссария
  reusable/             общие фрагменты для conref
  images/               рисунки
```

Папку можно создать прямо в программе: кнопка «Папка» на вкладке «Проект» или пункт «Новая папка…» в меню дерева (вложенная — через слеш, например `tasks/new`); пустые папки в дереве видны.

Рисунок 7. Окно «Новая папка»



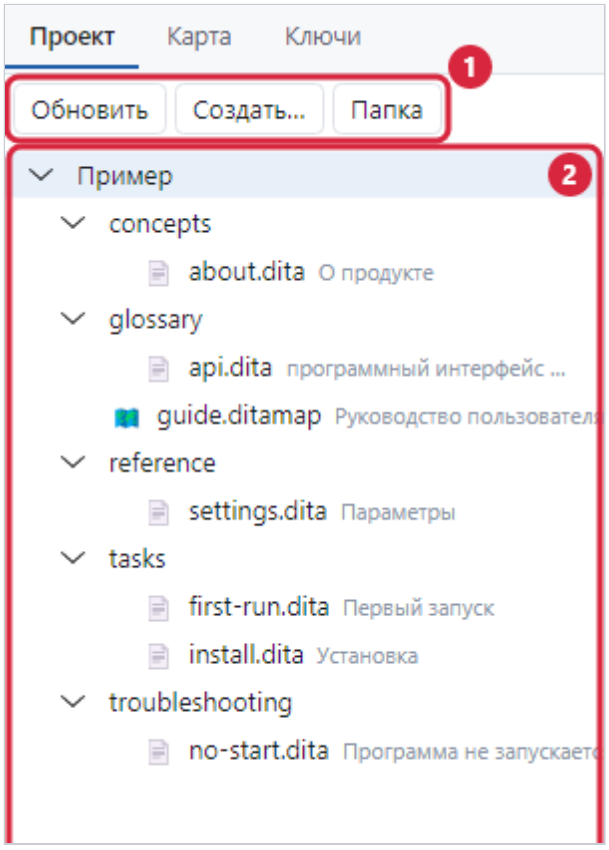
Скрытые папки и папки `out`, `bin`, `obj`, `temp`, `node_modules` редактор не сканирует. Сборку он по умолчанию предлагает класть в `out`.

2. **Откройте папку: Файл → Открыть папку проекта… (Ctrl+Shift+O).**

   →

   Пустая папка — нормальный проект: дерево на вкладке «Проект» пока пустое. Над деревом — кнопки «Обновить», «Создать…» и «Папка» (1), ниже — папки и файлы с заголовками топиков (2).

Рисунок 8. Вкладка «Проект» учебного проекта



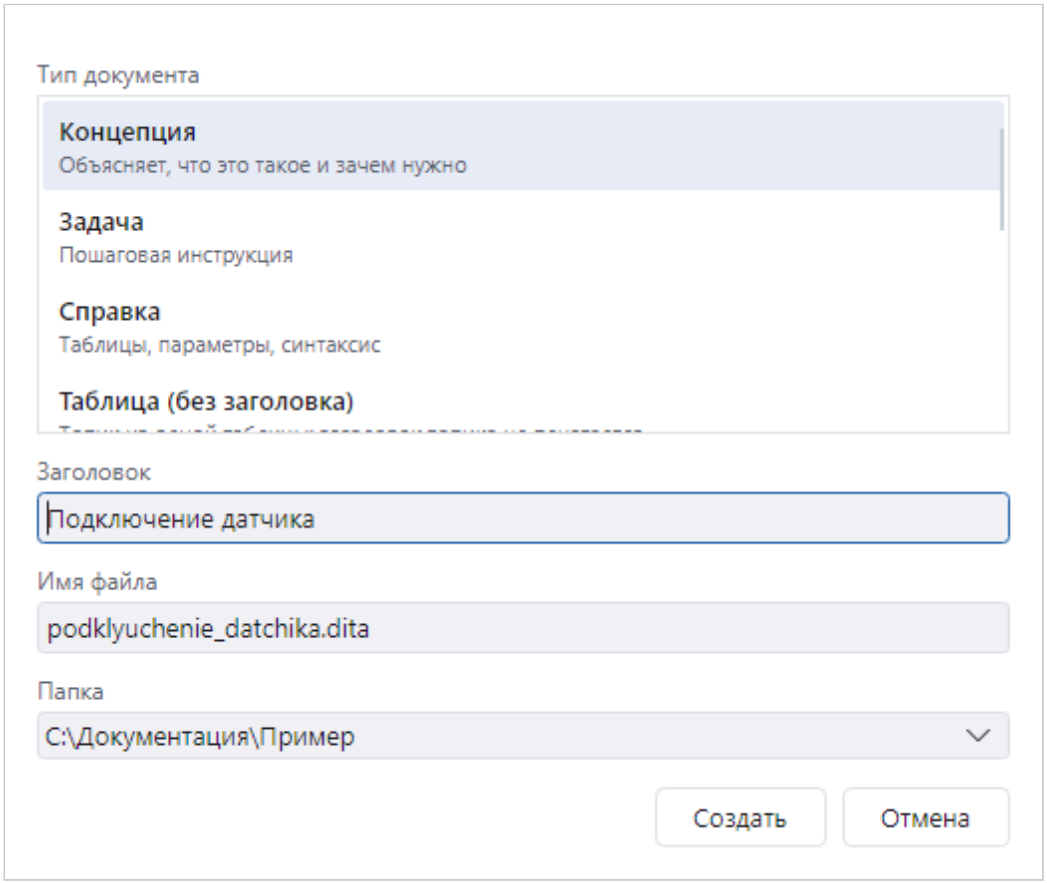
3. **Создайте карту: Файл → Создать документ... → Карта.**

   Заголовок — название будущей публикации, папка — корень проекта. Для книги с главами и приложениями выберите **Карта книги**, см. Карты публикации.

4. **Создайте первые топики через Ctrl+N, выбирая тип по содержанию.**

Если вы пишете...	**Выберите**
что это и зачем	Концепция
как сделать, по шагам	Задача
параметры, коды, команды	Справка
не работает — почему — что делать	Устранение неполадки
определение термина	Статья глоссария

Рисунок 9. Окно «Создать документ»



Имя файла и `id` строятся из заголовка (кириллица транслитерируется).

Когда карта уже есть, новый топик сразу добавляется в карту, выбранную на вкладке **Карта**, — после выделенной там строки (без выделения — в конец). Карта при этом открывается во вкладке с несохранённой правкой — сохраните её.

→

Документ открывается в режиме «Автор» с заготовкой. Заполните краткое описание — одно-два предложения о том, что даёт топик, — и основную часть. Подробно о типах: Типы топиков.

5. **Добавьте в карту топики, созданные до неё, и расставьте порядок на вкладке Карта.**
   - **+ Топик** добавляет ссылку на файл после выделенной строки или в конец карты.
   - **+ Раздел** добавляет заголовок раздела без своей страницы; его текст правится в `navtitle`, если открыть карту как документ.
   - → вкладывает строку в предыдущую, ← выносит, ↑ / ↓ и перетаскивание меняют порядок.
   - Флажок у строки — «публиковать»: снимите его, и топик вместе с веткой под ним не попадёт ни в HTML, ни в PDF, ни в DOCX (строка станет перечёркнутой). Файл и ссылки на него остаются, ключи и повторно используемые фрагменты из него работают.

- Правая кнопка мыши на строке открывает меню: открыть топик, свойства строки, добавить топик перед, после или внутрь, дублировать, найти ссылки на топик, вырезать, копировать и вставить строки, переименовать или удалить файл, развернуть или свернуть всё дерево.

→

Порядок строк карты — это порядок и вложенность страниц в публикации.

6. **Заведите ключи для того, что повторяется и может измениться: название продукта, версию, адрес сайта.**

   Откройте карту, перейдите в режим «Исходный код» и добавьте определения:

   ```
   <keydef keys="product">
     <topicmeta><keywords><keyword>МойПродукт</keyword></keywords></topicmeta>
   </keydef>
   <keydef keys="install" href="tasks/install.dita"/>
   ```

   После сохранения ключи появятся на вкладке «Ключи». Как их использовать — Ключи и ключевые слова.

7. **Проверьте проект: F7.**

   Ошибки (недопустимый порядок элементов, битые ссылки, необъявленные ключи, повторяющиеся `id`) исправьте до публикации. Двойной щелчок по строке переходит к месту ошибки.

8. **Посмотрите результат на вкладке Предпросмотр и соберите сайт: F5.**

3.1.2 Результат

В выбранной папке появится HTML-сайт с оглавлением. Тем же меню **Публикация** собираются единый HTML, PDF и DOCX.

3.1.3 Дальнейшие действия

Советы по устройству проекта:

- **Один топик — одна мысль.** Топик должен читаться отдельно от соседей. Если заголовок приходится писать через «и», это, скорее всего, два топика.
- **Стабильный `id`.** На него ссылаются `href` и `conref`; меняйте его через Структура → Переименовать id... — ссылки обновятся сами.
- **Имя файла совпадает с `id`.** Переносите файлы правым щелчком на вкладке «Проект», а не в Проводнике, — тогда редактор обновит ссылки.
- **Контроль версий.** Папку сборки `out` в репозиторий не добавляйте. Файлы `.ditastudio-*` — настройки проекта; добавьте их, если настройки нужны всем, но проверьте, что пути в них не абсолютные.

3.2 Открытие и создание проекта

Как открыть папку с DITA-проектом, быстро вернуться к недавнему проекту и создать новый документ по шаблону.

3.2.1 Контекст

Проект в DITA Studio — это папка на диске: одна или несколько карт (`.ditamap`) и топики рядом с ними. Открыть проект можно через Файл → Открыть папку проекта....

Порядок действий

1. **Откройте проект: Файл → Открыть папку проекта... (Ctrl+Shift+O) и выберите папку, где лежат карта и топики.**
2. **Чтобы быстро вернуться к одному из последних проектов, откройте Файл → Недавние проекты и выберите папку из списка.**

   Список хранит до восьми последних папок и переживает перезапуск редактора; папки, которые больше не существуют на диске, в списке не показываются.
3. **Чтобы работать сразу с несколькими проектами, откройте ещё одну папку тем же пунктом меню: она добавится рядом с уже открытой, а вкладки первого проекта останутся.**

   Активен проект выбранной вкладки или файла в дереве; его название — в заголовке окна. Одноимённые топики разных проектов различаются по названию проекта в заголовке вкладки. Закрыть один проект можно командой Файл → Закрыть проект — остальные проекты не затрагиваются. Ссылки между проектами не разрешаются.
4. **Чтобы создать документ, выберите Файл → Создать документ... (Ctrl+N).**

Шаг диалога	Что происходит
Тип документа	Выбирается заготовка: концепция, задача, справка, устранение неполадки, статья глоссария или карта.
Заголовок	Имя файла и `id` корневого элемента предлагаются автоматически из заголовка (кириллица транслитерируется).

3.2.2 Результат

Проект открыт, дерево файлов слева и список карт на вкладке «Карта» заполнены — можно переходить к редактированию содержимого.

3.3 Редактирование текста и структуры

Основные приёмы работы в режиме «Автор»: набор текста, подсказка по Enter, оформление-переключатели, выделение и перенос блоков, обёртывание, пометки атрибутов.

3.3.1 Контекст

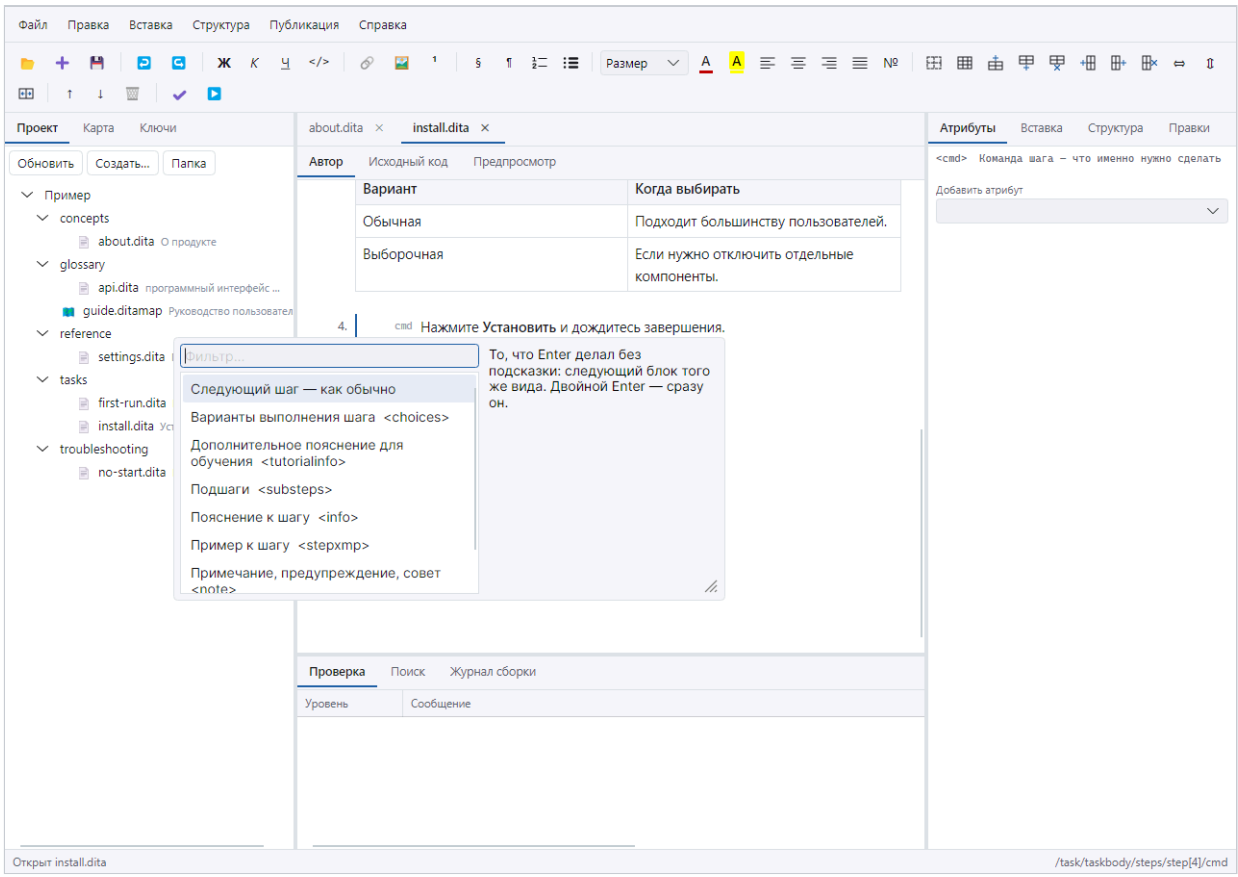
Откройте любой топик проекта и переключитесь на вкладку **Автор** над областью редактирования. Каждый блок (`p`, `li`, `cmd`, `title`…) — отдельная строка редактирования; контейнеры (`section`, `div`, `fig`, `note`) обведены полосой слева с подписью.

Порядок действий

1. **Наберите текст абзаца и нажмите Enter в конце блока.**

   Открывается подсказка: первая строка — обычное действие (следующий абзац, шаг, пункт), ниже — все элементы, допустимые после блока по контент-модели, а в конце пункта списка или раздела — и после самого контейнера. Справа — описание из каталога. Набор текста фильтрует список, двойной Enter выбирает первую строку. Ctrl+Enter открывает то же меню для вставки **после внешнего** блока — так выходят из таблицы, `div`, списка.

Рисунок 10. Подсказка по Enter в конце команды шага



Окно растягивается за любой угол, размер запоминается. Выключить подсказку — Структура → Подсказка по Enter.

2. **Enter в середине текста разбивает блок по позиции курсора, Backspace в начале блока объединяет его с предыдущим.**

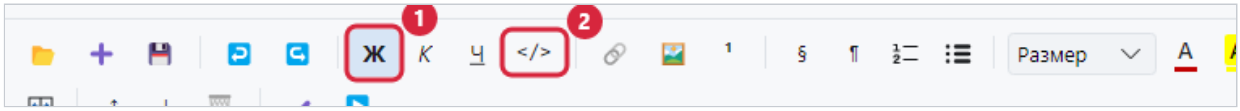
   Обе операции идут через `EditCommands` и не выполняются, если результат нарушил бы контент-модель; причина — в строке состояния.

3. **В списке или в шагах задачи Tab и Shift+Tab меняют уровень вложенности — например, превращают шаг в подшаг (`substeps`).**

4. **Оформите текст: Ctrl+B — `b`, Ctrl+I — `i`, Ctrl+U — `u`, Ctrl+` — `codeph`, Ctrl+Shift+U — `uicontrol`.**

   Кнопки и сочетания — переключатели: когда курсор или выделение в оформленном тексте, кнопка подсвечена, повторное нажатие снимает элемент — с выделенной части (элемент разрезается) или со всего элемента под курсором. Поле **Размер**, полоски цвета и маркера и кнопки выравнивания показывают оформление под курсором. Ctrl+Shift+Space снимает всё оформление.

Рисунок 11. Курсор в полужирном тексте: «Ж» нажата (1); кнопка codeph (2)



   Размер, цвет, маркер и выравнивание пишутся классами `outputclass` (`size-10`, `color-red`, `mark-yellow`, `align-center`), которые публикация переводит в оформление; CSS проекта может их переопределить.

5. **Выделите блок целиком: щёлкните по его подписи, рамке или полю вокруг, а не по тексту.**

   Блок обводится контуром; Delete удаляет его одним шагом отмены, Esc снимает выделение. Протяжка мышью из одного блока в другой выделяет диапазон блоков (у вложенных берётся внешний), Shift+щелчок продлевает его.

6. **С выделенными блоками: Ctrl+C, Ctrl+X, Ctrl+V, Enter — «Обернуть в...».**

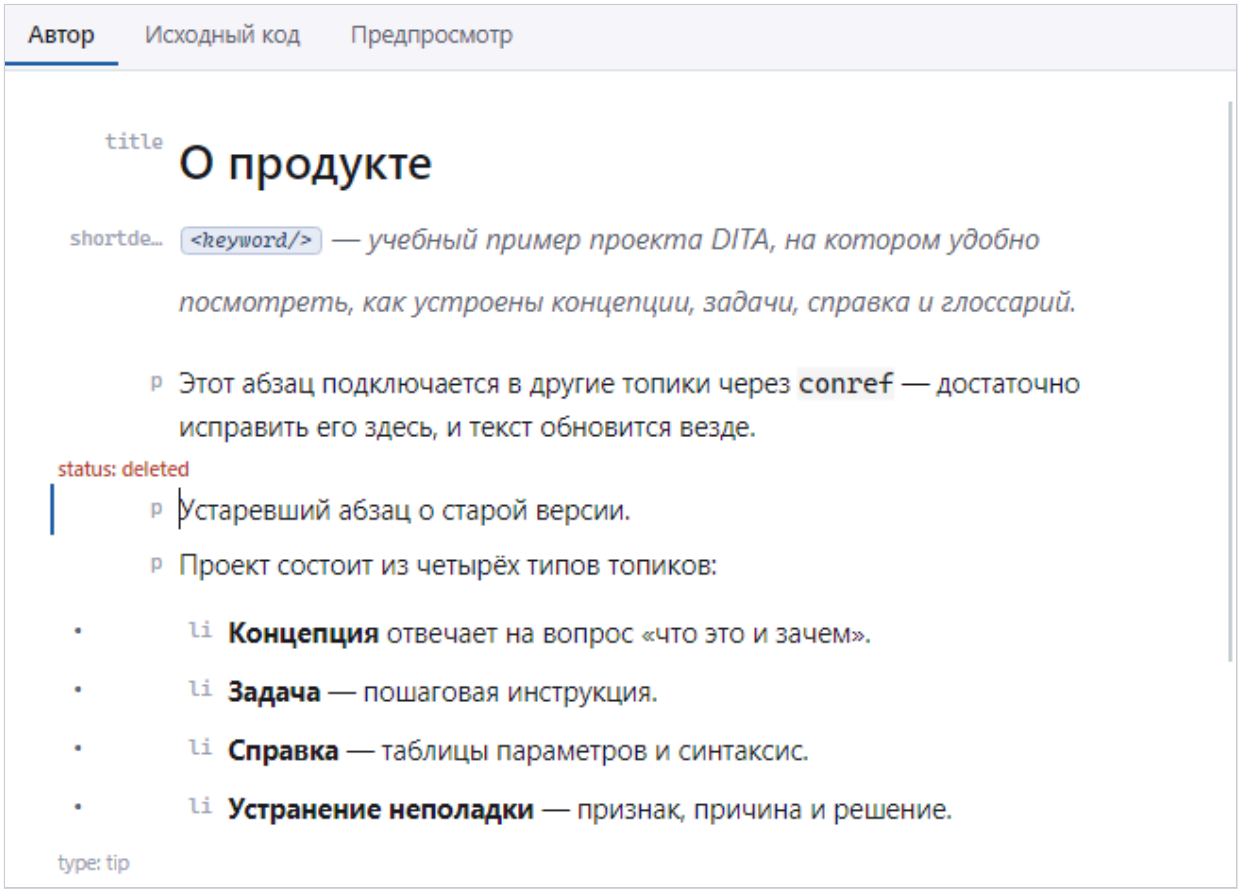
   В буфер идёт XML блоков (его можно вставить и в «Исходный код»). Вставка проверяется по контент-модели: если сразу за блоком нельзя, блоки встают после ближайшего подходящего предка; повторяющиеся `id` у копий заменяются. «Обернуть в...» предлагает только допустимые обёртки (`note`, `section`, `div`, `fig`, `example`...).

7. **Переставьте блок без мыши: Ctrl+Shift+↑ / Ctrl+Shift+↓.**

8. **Следите за атрибутами по серым пометкам над блоками.**

   Над блоком с атрибутами — строка вида «product: Альфа · rev: 2 · class: warning-box»: сначала условные атрибуты, затем остальные, последним `outputclass`. Пометка `status` цветная: `new` — зелёная, `changed` — оранжевая, `deleted` — красная. Выключить — Структура → Показывать пометки атрибутов.

Рисунок 12. Пометка status: deleted над абзацем



3.3.2 Результат

Документ остаётся валидным на каждом шаге — редактор просто не даст выполнить операцию, которая сломает структуру. Заодно работает проверка орфографии по словарям русского и английского языков: слова с ошибкой подчёркиваются волнистой линией, правый клик по слову предлагает варианты замены и команду **Пропустить все**. Код, пути к файлам и имена команд не проверяются.

Изменили заголовок топика — редактор предложит переименовать файл по нему; ссылки на файл обновятся.

3.3.3 Дальнейшие действия

Про вставку изображений и ссылок — в задаче Вставка изображений, ссылок и сносок, про таблицы — Таблицы в режиме «Автор».

3.4 Вставка изображений, ссылок и сносок

Палитра допустимых элементов, рисунки и изображения в строке, свойства и размер картинки, перекрёстные ссылки, ключи и сноски.

3.4.1 Перед началом

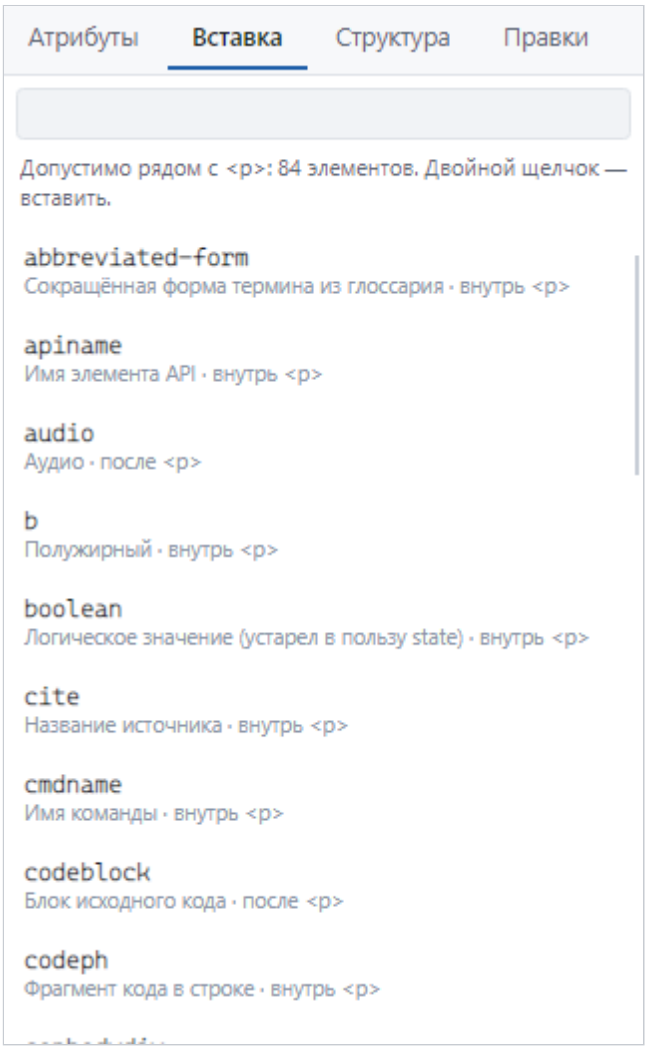
Всё, что можно вставить у курсора, собрано и в контекстном меню режима «Автор»: щёлкните правой кнопкой мыши в тексте и откройте **Вставить элемент**. Там частые вставки (сноска, изображение, ссылка, таблица) и все допустимые в этом месте элементы — **В строку текста** и **Блок после текущего**.

Порядок действий

1. **Палитра: откройте вкладку Вставка справа (Ctrl+E).**

   В палитре — только элементы, которые автомат контент-модели допускает в текущей позиции, с пометкой «внутрь <p>» (фразовый — на место курсора, выделенный текст оборачивается) или «после <p>» (блок после текущего). Двойной щелчок — вставить. Элементы подключённого DTD появляются здесь же.

Рисунок 13. Палитра вставки для курсора в абзаце



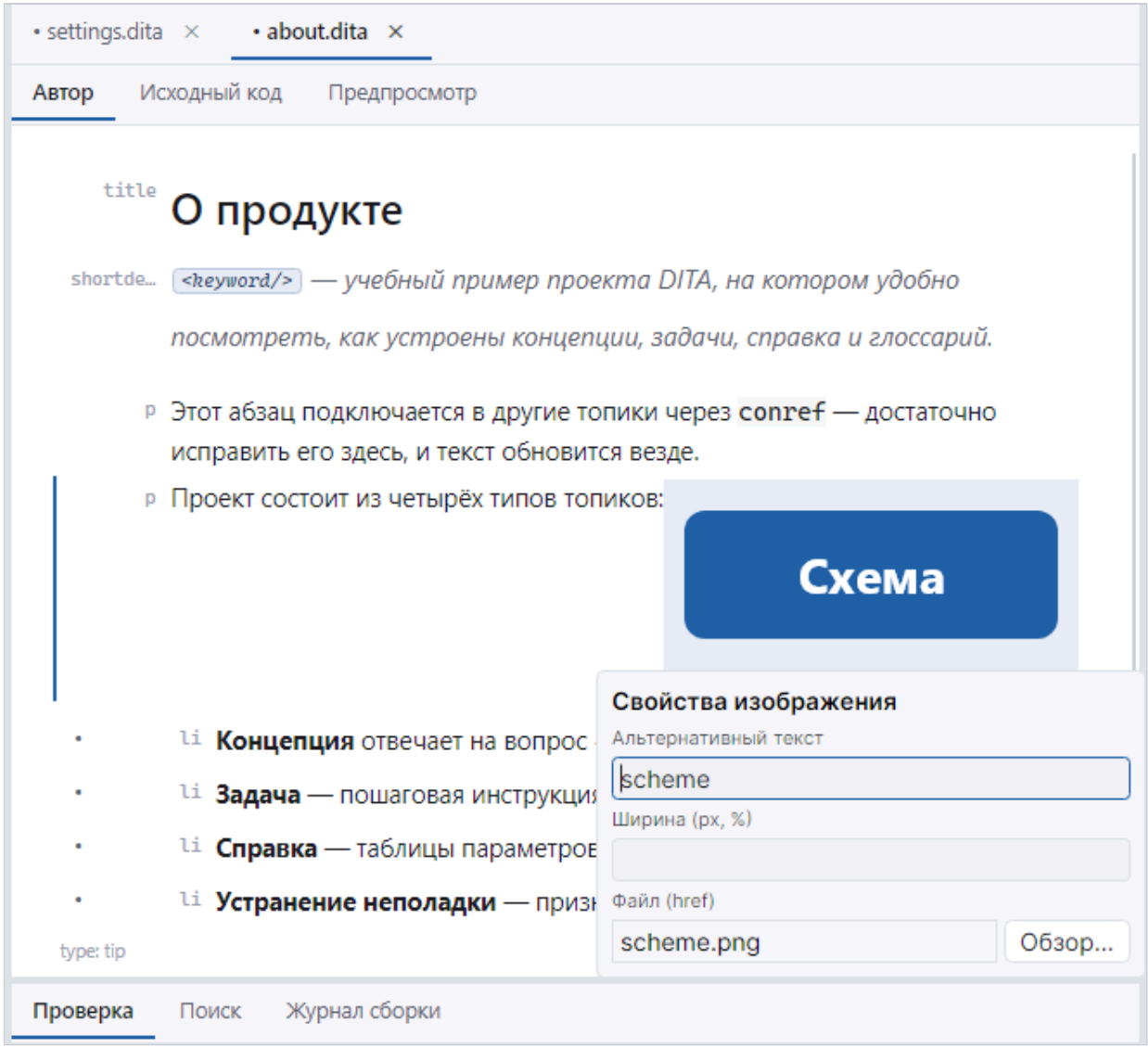
2. **Изображение: Вставка → Изображение….**

   Сначала вопрос: **Да** — рисунок `fig` с названием и номером после текущего блока, **Нет** — `image` в строке у курсора. Относительный `href` подставляется сам, заготовку `alt` нужно заполнить. Вставка → Оформить изображение как рисунок переносит уже стоящий в абзаце `image` в `fig`.

3. **Свойства изображения: щёлкните по картинке.**

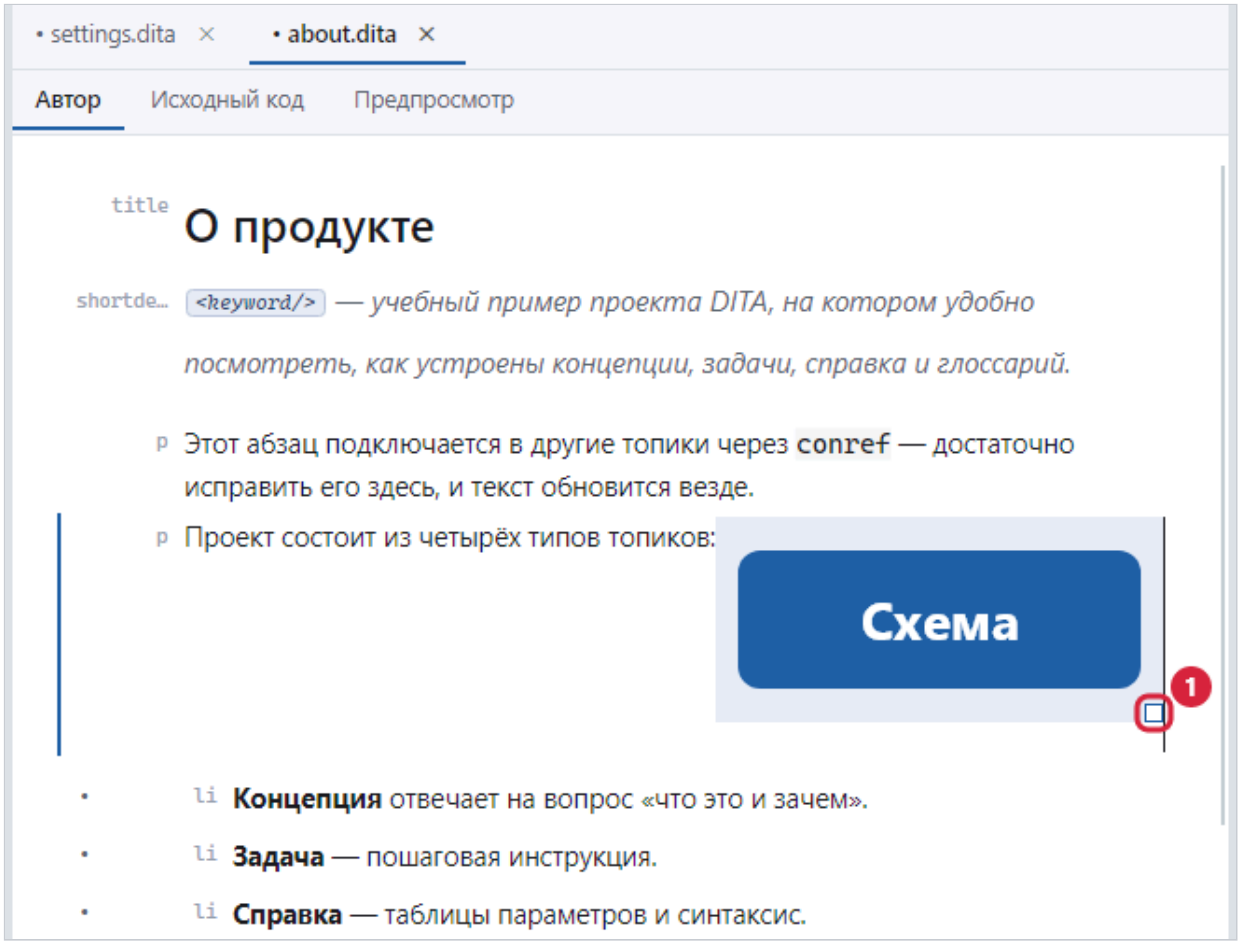
   Под картинкой открывается узкое окно: `alt`, ширина (`320`, `320px`, `50%`) и файл (`href`; длинный путь сокращается многоточием, полный — в подсказке, **Обзор…** выбирает другой). Enter или щелчок мимо записывают, Esc отменяет; запись — одна правка.

Рисунок 14. Окно «Свойства изображения»



4. **Размер мышью: потяните за квадратный маркер в правом нижнем углу картинки.**

Рисунок 15. Маркер размера (1)



Пропорции сохраняются; пишется `width` в пикселях, `height` снимается. Работает в любом блоке — ячейке таблицы, примечании, пункте списка; шире своего блока картинку не растянуть, а протаскивание маркера не начинает выделение ячеек. Название рисунка — правая кнопка по названию → **Наименование рисунка…**.

5. **Перекрёстная ссылка: Вставка → Перекрёстная ссылка… — список топиков проекта с фильтром по названию.**

   Щелчок по плашке ссылки без текста открывает «Свойства ссылки»: текст, `href` и `scope`.

6. **Ключ (`keyref`): опишите значение один раз в `keydef` карты, а затем ссылайтесь на него из любого топика через `<keyword keyref="…"/>` или через `keyref` на пустом элементе — как DITA Studio в этом руководстве.**

   Текст ключа подставляется только в пустой элемент: если написать текст внутри тега с `keyref`, он останется как есть и ключ подставлен не будет. Подробнее — Ключи.

7. **Сноска (`fn`): выберите Вставка → Сноска — на месте курсора появится плашка сноски, а курсор перейдёт в её строку в области Сноски внизу топика с выделенной заготовкой «Текст сноски»; наберите текст сноски, он заменит заготовку. Если перед вставкой выделить текст, он станет текстом сноски.**

   На сайте сноски печатаются в конце того топика, где они поставлены; в DOCX это настоящие сноски Word на своей странице.

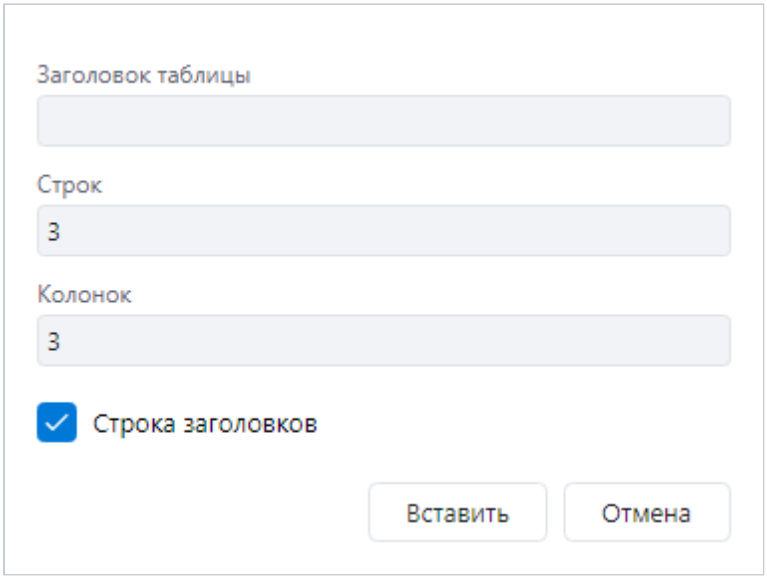
3.5 Таблицы в режиме «Автор»

Таблицы CALS правятся как в текстовом процессоре: вставка, выделение ячеек мышью, строки и столбцы, объединение, границы, ширина столбцов, название, обмен ячейками с Excel. В файл пишутся стандартные атрибуты CALS.

Порядок действий

1. **Вставьте таблицу: Вставка → Таблица….**

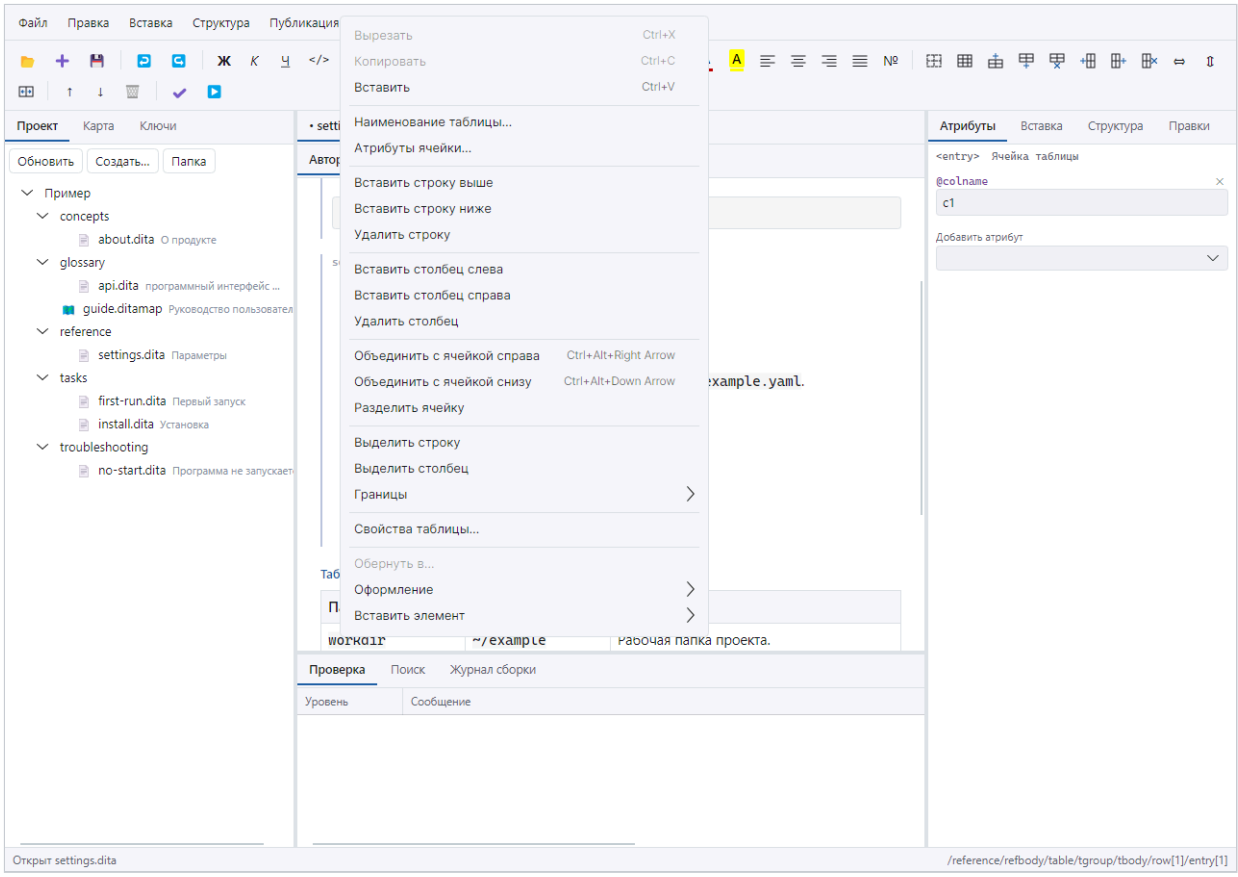
Рисунок 16. Окно вставки таблицы



   Пустой заголовок — таблица без `title` (без подписи и номера). Флажок «Строка заголовков» создаёт `thead`: она повторяется на каждой странице PDF и DOCX у таблицы с классом `page-break-auto`.

2. **Откройте меню ячейки правой кнопкой мыши.**

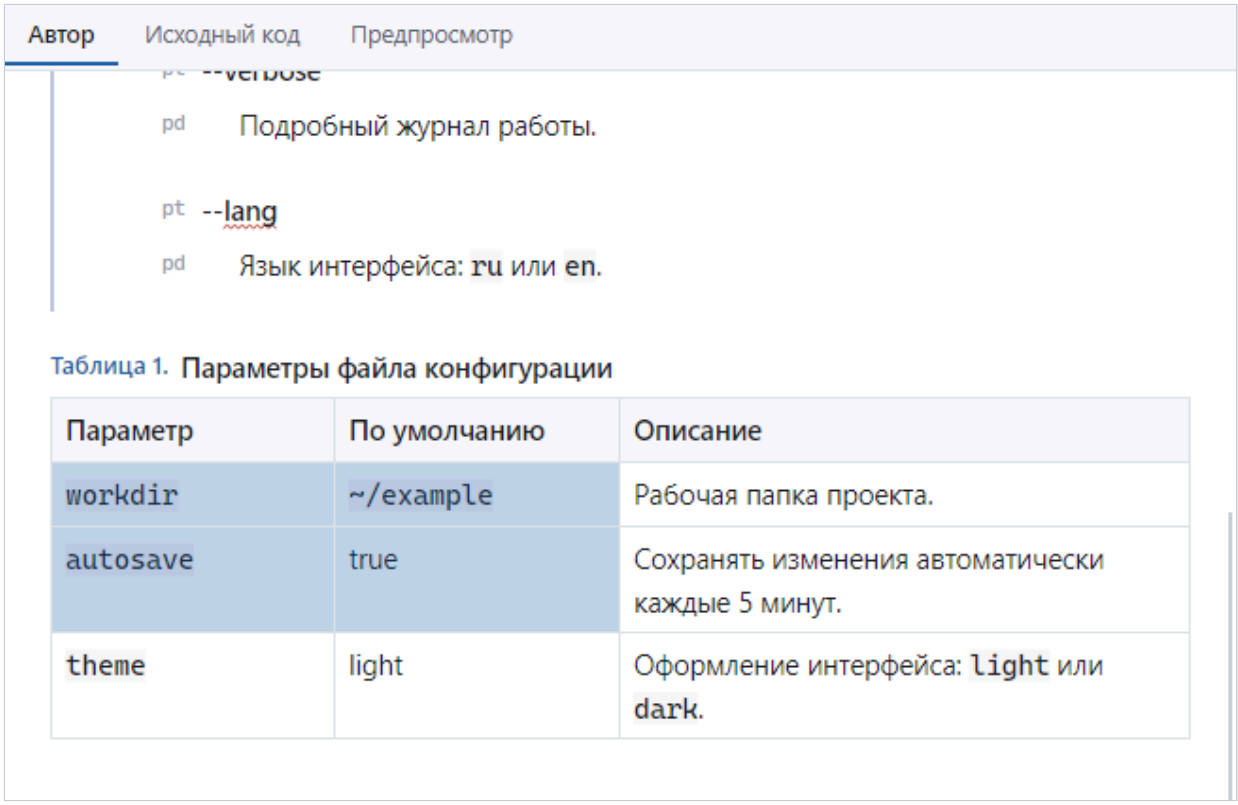
Рисунок 17. Меню ячейки



Объединённые ячейки учитываются: строка посреди растянутой вниз ячейки удлиняет её (`morerows`), при удалении строки растянутая ячейка переезжает в следующую с текстом, столбец внутри горизонтального объединения его расширяет (`namest`/`nameend`). У `simpletable` работают строки и столбцы (`relcolwidth` и `keycol` пересчитываются), у `properties` и `choicetable` — только строки. Ctrl+Alt+→ и Ctrl+Alt+↓ объединяют ячейку с соседней справа и снизу.

3. **Выделите прямоугольник ячеек протяжкой мыши (или Shift+щелчком).**

Рисунок 18. Выделенные ячейки



Меню выделения вставляет и удаляет столько строк и столбцов, сколько выделено, объединяет ячейки (содержимое собирается в левой верхней), очищает, выравнивает и ставит границы. Объединить нельзя через границу `thead` и `tbody` и так, чтобы у строки не осталось ячеек.

Ctrl+C/Ctrl+X/Ctrl+V работают с ячейками: в буфер идёт текст с табуляциями (как в Excel), внутри редактора сохраняется разметка ячеек. Вставка прямоугольником от левой верхней ячейки; нехватка строк удлиняет таблицу. Текст с табуляциями вне таблицы вставляется новой таблицей.

4. **Задайте границы: значок «Границы» на панели или Структура → Таблица → Границы выделенных ячеек.**

   Нижняя, верхняя, левая, правая, нет границы, все, внешние, внутренние — как в Word, пункты переключаются. Линия между ячейками — `rowsep`/ `colsep` ячейки, край таблицы — `frame`; нестандартная рамка пишется классами `frame-top`, `frame-bottom`, `frame-left`, `frame-right`. Вид линий (толщина, цвет) задаёт CSS — см. Оформление DOCX.

5. **Ширина столбца и высота строки — протяжкой границы мышью.**

   Ширина пишется долями (`colwidth="75*"`, `relcolwidth`), высота — классом `row-height-15mm`; двойной щелчок по нижней границе строки возвращает высоту по содержимому.

6. **Название: Наименование таблицы… в меню ячейки.**

   Нет `title` — он создаётся; пустое поле очищает название, подпись «Таблица N» остаётся. Вставка → Подпись рисунка / таблицы: добавить или убрать добавляет или убирает сам `title`.

3.5.1 Результат

Щелчок по полю над таблицей или под ней (вне ячеек) выделяет таблицу целиком — Delete удаляет её одним шагом отмены. Выделение ячеек и границы доступны только у таблиц CALS (`table`), не у `simpletable`.

3.6 Проверка проекта и поиск

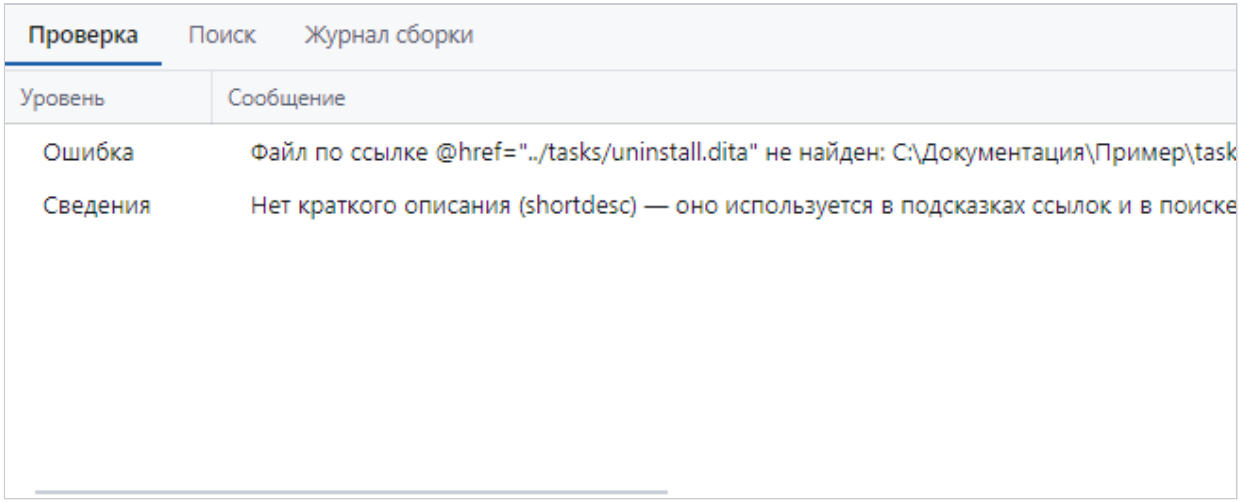
Как найти ошибки структуры и текст по всему проекту, не открывая каждый файл по отдельности.

Порядок действий

1. **Нажмите F7 или выберите Правка → Проверить проект, чтобы проверить сразу все файлы проекта; для одного открытого документа есть отдельная команда «Проверить документ».**

   → Список найденных проблем открывается на отдельной вкладке снизу, с переходом к нужному месту по двойному щелчку.

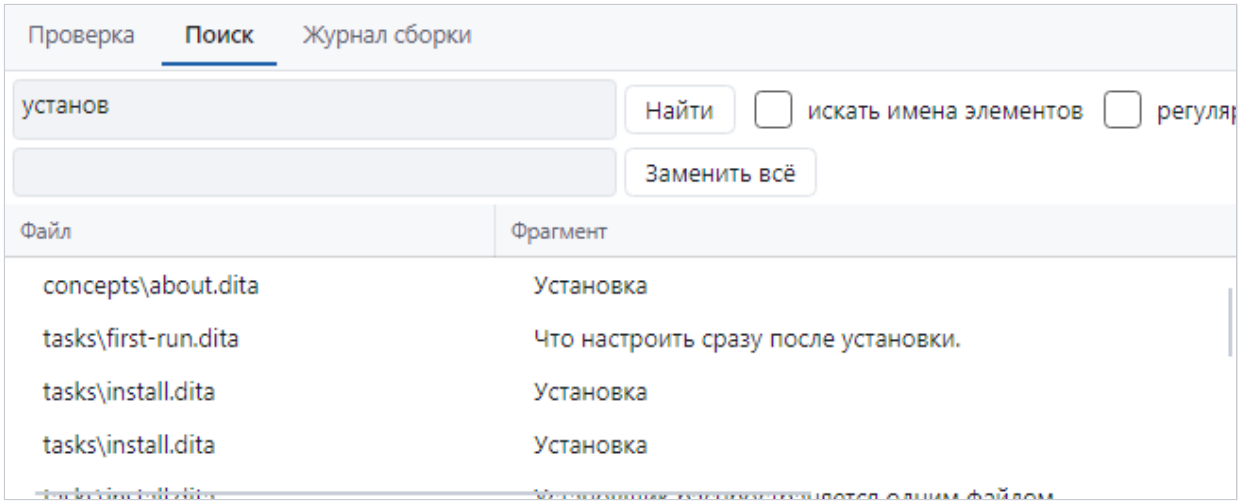
Рисунок 19. Вкладка «Проверка»



2. **Проверяются: соответствие модели содержимого и допустимые атрибуты (включая значения перечислений), повторяющиеся `id`, а также редакторские рекомендации — пустой заголовок или шаг, отсутствующее краткое описание, изображение без `href`/`keyref` или без `alt`, битые перекрёстные и key-ссылки.**

3. **Чтобы найти текст по всему проекту, нажмите Ctrl+Shift+F — можно искать как обычный текст, так и по именам элементов; галочка «регулярное выражение» включает поиск по regex (синтаксис .NET) вместо буквального совпадения.**

Рисунок 20. Вкладка «Поиск»



4. **Чтобы заменить найденное сразу во всём проекте, впишите текст замены во второе поле и нажмите Заменить всё.**

   Замена работает по отдельным текстовым узлам и не пересекает границу с вложенными элементами (например, если часть фразы оформлена **полужирным**) — она не заменяет имена элементов и недоступна при включённом поиске по именам элементов. Изменённые документы помечаются как несохранённые — сохраните их (Ctrl+Shift+S) после проверки результата.

3.7 Условия сборки и публикация

Как исключить часть содержимого из публикации по атрибутам условности и собрать итоговый HTML, PDF или DOCX.

3.7.1 Контекст

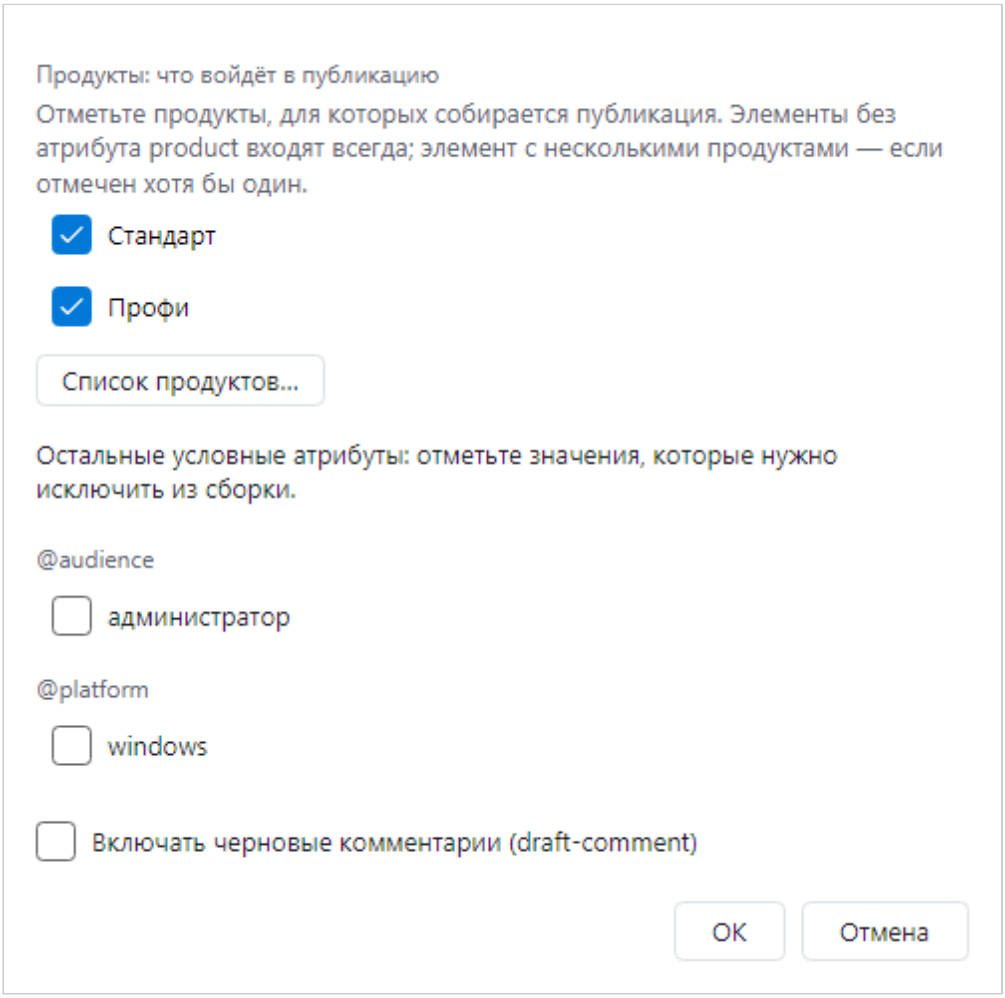
DITA Studio умеет собирать один и тот же проект по-разному в зависимости от значений атрибутов `props`, `platform`, `product`, `audience`, `otherprops` и `deliveryTarget`, расставленных в топиках. Условия задаются в диалоге, в стандартном файле `.ditaval` или обоими способами сразу.

Порядок действий

1. **Откройте Публикация → Условия сборки...**

   → Вверху — продукты проекта флажками: отмечены те, для которых собирается публикация (элементы без `product` входят всегда, элемент с несколькими продуктами — если отмечен хотя бы один). Список продуктов правится в Публикация → Список продуктов... и переносится в другой проект импортом; значение `product` выбирается флажками на панели **Атрибуты**. Ниже — остальные условные атрибуты: отметьте значения, которые нужно исключить, и при необходимости включите показ черновых комментариев (`draft-comment`).

Рисунок 21. Окно «Условия сборки»



2. **Подтверждённые условия сохраняются вместе с проектом и применяются автоматически при следующей сборке — переоткрывать диалог заново после перезапуска редактора не нужно.**

3. **Если условия уже описаны в стандартном файле `.ditaval` (например, из другого инструмента), выберите Публикация → Подключить .ditaval....**

   Файл не копируется, а перечитывается при каждой сборке — правки, сделанные в нём снаружи, подхватываются автоматически. Правила `action="exclude"` складываются с отмеченными в диалоге, правила `action="flag"` подсвечивают содержимое в публикации (цвет, фон, начертание, полоса изменений). Правило `action="exclude"` без `val` исключает все значения атрибута, кроме возвращённых правилами `action="include"`; `action="passthrough"` выводит значение атрибута в HTML как `data-`атрибут. Условия действуют и на строки карты: исключённый `topicref` уходит из публикации вместе с веткой.

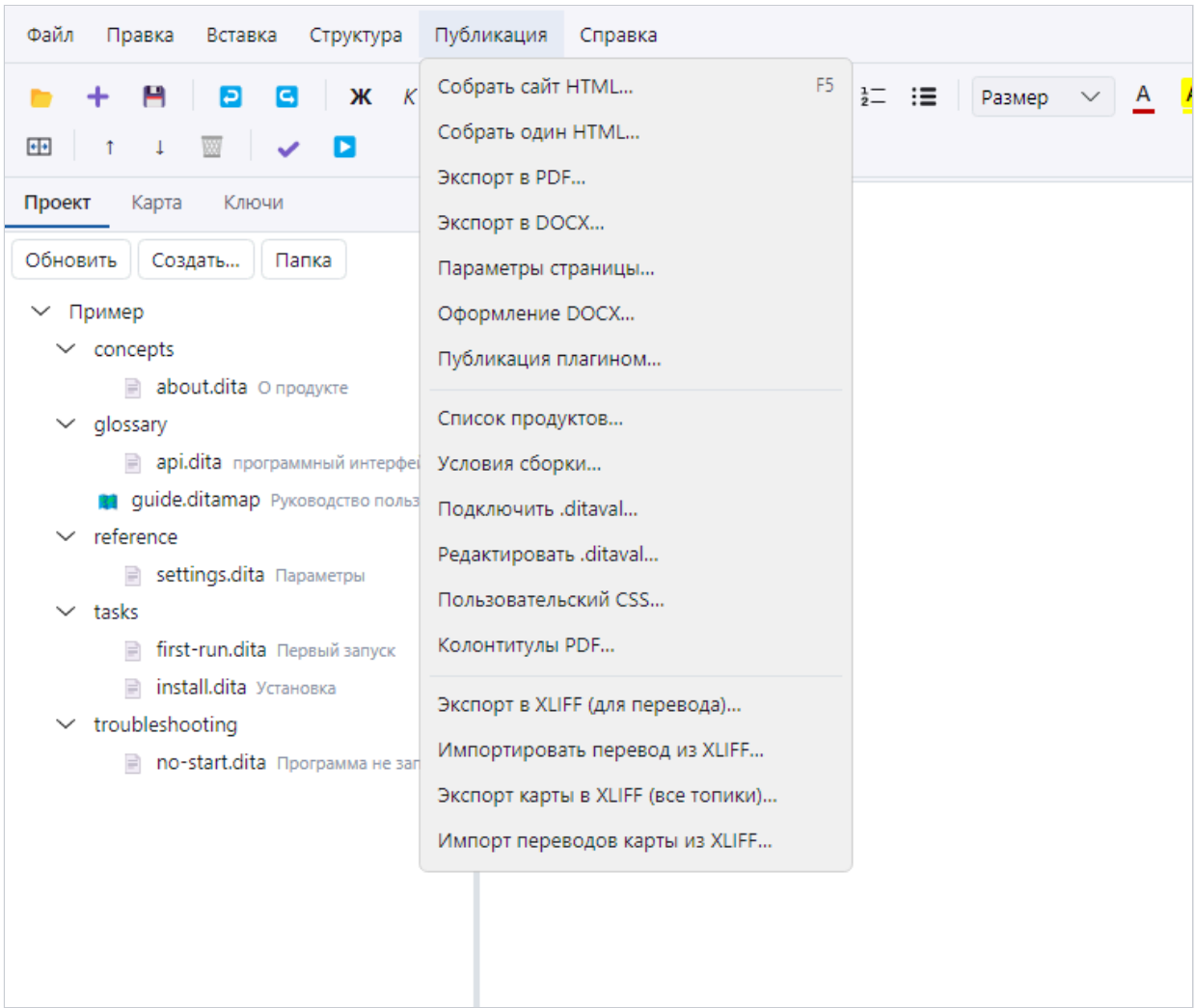
4. **Чтобы изменить исключения в связанном файле, не выходя из редактора, выберите Публикация → Редактировать .ditaval....**

→ Отмеченные значения записываются обратно в тот же файл; правила подсветки в нём не меняются.

5. **Выберите способ публикации в меню Публикация: «Собрать сайт HTML» (F5) — многостраничный сайт с оглавлением, «Собрать один HTML» — всё содержимое карты в одном файле с предметным указателем, «Экспорт в PDF» — тот же HTML, напечатанный в PDF, «Экспорт в DOCX» — документ Word с настоящими сносками и полем оглавления.**

   Перед сборкой несохранённые документы сохраняются автоматически. Предупреждения сборки (битые ссылки, SVG, который не удалось отрисовать для DOCX) — на вкладке «Журнал сборки». Как оформить документ Word — Оформление DOCX.

Рисунок 22. Меню «Публикация»



6. **По умолчанию PDF печатается без колонтитулов. Чтобы включить их или задать свой текст, откройте Публикация → Колонтитулы PDF...: отметьте Показывать колонтитулы и впишите текст шапки и подвала — номер страницы и их общее число добавляются автоматически.**

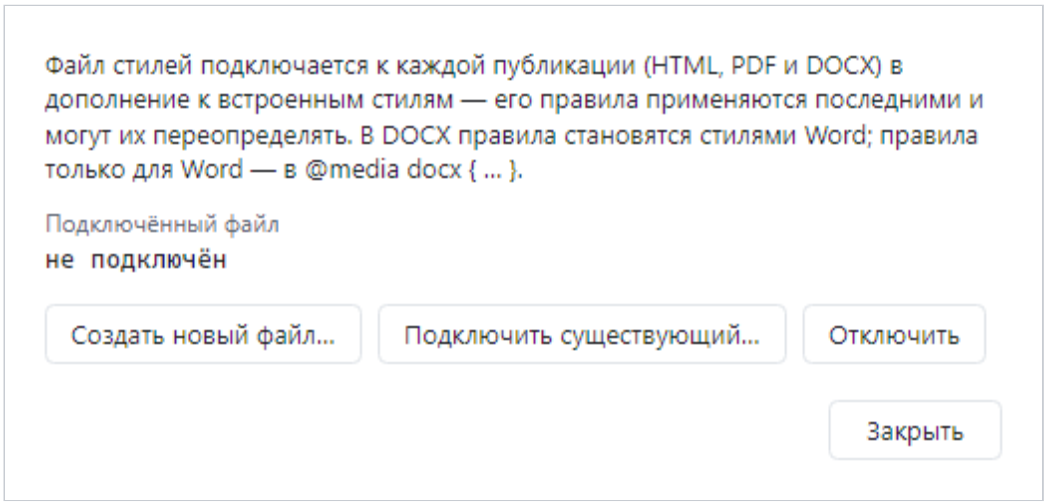
С колонтитулами PDF печатает встроенный браузер редактора: текст шапки — по центру, текст подвала — слева, номер страницы — справа. Без колонтитулов печатает установленный Microsoft Edge или Google Chrome, а если его нет — тоже встроенный браузер.

7. **Разрыв страницы перед заголовком и перенос таблицы между страницами задаются через атрибут `outputclass` в панели «Атрибуты» (или пунктами меню «Структура»): значение `page-break-before` на `title` и `page-break-auto` на `table`.**

   Заголовок этой задачи начинает новую страницу печати именно так — можно проверить, открыв документ в режиме **Исходный код**.

8. **Чтобы изменить оформление публикации, подключите файл стилей через Публикация → Пользовательский CSS... — путь к файлу тоже сохраняется вместе с проектом.**

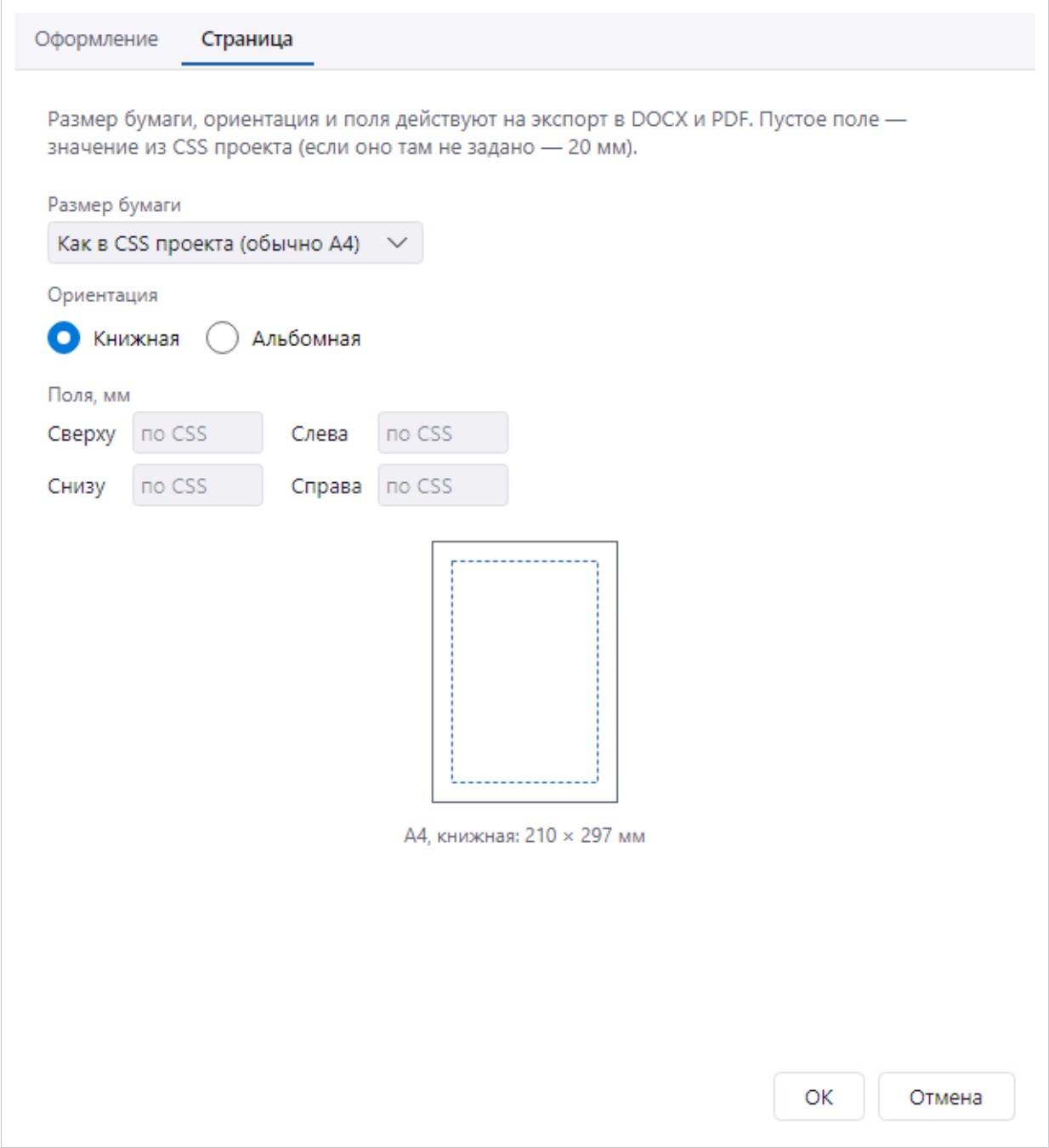
Рисунок 23. Окно «Пользовательский CSS»



«Создать новый файл...» записывает заготовку с готовыми классами разрывов страниц и примерами. Справочник селекторов и свойств — `docs/CSS_REFERENCE.md` в репозитории редактора.

9. **Размер бумаги, ориентацию и поля для PDF и DOCX задайте в Публикация → Параметры страницы....**

Рисунок 24. Параметры страницы



Пустое поле — значение из `@page` пользовательского CSS (без него A4 с полями 20 мм); заданное в окне главнее CSS.

3.7.2 Результат

Готовая публикация лежит в выбранной папке вывода; при ошибке экспорта в PDF см. PDF не экспортируется.

3.7.3 Оформление DOCX

Внешний вид документа Word задаётся тем же пользовательским CSS, что и сайт, а устройство документа — титул, оглавление, нумерация заголовков,

колонтитулы — диалогом «Оформление DOCX».

Стили Word из CSS

Экспорт в DOCX не форматирует текст напрямую: каждому виду содержимого соответствует именованный стиль Word, а правила пользовательского CSS меняют эти стили. Документ можно дорабатывать и в самом Word: изменение стиля на вкладке «Главная» разом меняет весь документ.

Селектор CSS	Стиль Word
`body`	«Обычный» — только свойства текста: шрифт, размер, цвет
`p`	«Основной текст» — абзацы, включая поля и выравнивание
`h1` … `h6`	«Заголовок 1…6»
`.shortdesc`	«Краткое описание»
`.note`, `.note.warning`, `.note.danger`	«Примечание» и его варианты по типу
`pre`, `.codeblock`	«Блок кода»
`code`, `.codeph`, `.uicontrol`	символьные стили «Код в тексте», «Элемент интерфейса»
`table`, `th`, `td`	рамки, заливка шапки, поля ячеек, шрифт текста таблицы
`ul`, `ul ul`, `li::marker`	маркеры списков по уровням (`disc`, `circle`, `square`, `none` или текст в кавычках) и их цвет

Правило для своего класса применяется к элементам с таким значением `outputclass`: экспорт создаёт производный стиль, например «Основной текст · warning-box». Класс на разделе оформляет все его абзацы, класс на фразе — только её текст.

Переносятся шрифты, размеры, цвета, заливка, подчёркивание, прописные и капитель, выравнивание, отступы, межстрочный интервал, рамки и разрывы страниц. Размер и поля страницы задаёт правило `@page`, правила только для Word — блок `@media docx`, который браузер пропускает.

Размер бумаги (стандартный или свой — ширина и высота в миллиметрах), ориентацию и поля проще задать без CSS: Публикация → Параметры страницы…. Схема листа в диалоге сразу показывает результат; заданное там действует на DOCX и PDF и главнее правила `@page`, пустые поля берутся из CSS.

Колонтитулы тоже задаются в CSS — полями страницы `@top-left`, `@top-center`, `@top-right` и такими же `@bottom-…` внутри `@page`. В `content` работают строки, `counter(page)`, `counter(pages)`, `string(title)` и `url(картинка)`; у поля — шрифт, размер, цвет и линия `border-bottom` / `border-top`. Они перекрывают колонтитулы из диалога **Оформление DOCX** и попадают в DOCX и PDF. Картинка в колонтитуле по умолчанию подгоняется под его высоту.

```
@page { size: A4; margin: 20mm 15mm 20mm 25mm; }
@page {
  @top-right { content: string(title); font-size: 9pt; color: #666; }
  @bottom-center { content: "Стр. " counter(page) " из " counter(pages); font-size: 9pt; }
}
@media docx {
  body { font-family: Georgia; font-size: 12pt; }
  h1 { color: #1a5fb4; border-bottom: 1pt solid #1a5fb4; }
}
```

Кроме тегов и классов HTML-публикации в DOCX работают настоящие селекторы CSS по элементам DITA: имя элемента (`cmd`, `entry`), класс `outputclass`, атрибут (`note[type="warning"]`), потомок и ребёнок (`section > title`), соседи, `:first-child`, `:nth-child()`, `:not()`, а также текст `::before` и `::after`. Правила складываются каскадом по специфичности и порядку в файле.

```
ol > li:first-child { font-weight: bold; }
note[type="warning"] { background-color: #fff2cc; }
tbody > row:nth-child(even) > entry { font-size: 9pt; }
li::before { content: "▸ "; }
```

> **Примечание:** Скругления, тени, фоновые картинки и вёрстка веб-страницы в Word не переносятся. О пропущенных свойствах и селекторах сообщает «Журнал сборки». Шрифт должен быть установлен на компьютере, где открывают документ.

Заголовок без номера

Введение, предисловие или справочный раздел часто не нумеруют и не выносят в оглавление. Поставьте курсор в заголовок, щёлкните правой кнопкой мыши и выберите **Заголовок без номера (не в оглавлении)** (или то же в меню **Структура**). В режиме «Автор» у заголовка появится пометка «без номера · не в оглавлении»; при публикации в DOCX, едином HTML и PDF он не нумеруется и в оглавление не попадает. В файле это класс `outputclass="nonumber"`, а для целой ветки карты — стандартный атрибут `toc` со значением `no` у строки карты.

Блок на отдельном листе

Гриф «Для служебного пользования» внизу страницы, схема посередине листа, лист утверждения — такие блоки ставятся на свою страницу в нужную область. Поставьте курсор в абзац, рисунок, таблицу или заметку, щёлкните правой кнопкой мыши и выберите Оформление → Положение на листе (PDF, DOCX) (или то же в меню **Структура**), затем одну из девяти областей: вверху, посередине или внизу — слева, по центру или справа. Пункт **Обычное (в тексте)** возвращает блок в текст.

В режиме «Автор» над блоком появится пометка «Отдельный лист» с выбранной областью. В PDF и DOCX блок стоит на своей странице, текст после него продолжается со следующей; на сайте и в предпросмотре топика он остаётся на своём месте в тексте. Курсор в подписи рисунка или в ячейке таблицы ставит на лист весь рисунок или всю таблицу. Блок внутри

списка, таблицы или заметки на отдельный лист не ставится. В файле это класс, например `outputclass="place-bottom-right"`; в DOCX — отдельный раздел Word с вертикальным выравниванием страницы, колонтитулы и параметры листа те же.

Шаблон DOCX

В диалоге «Оформление DOCX» можно подключить свой файл Word (`.docx` или `.dotx`). Из него берутся стили, шрифты, цвета и значения по умолчанию; стили, которых в шаблоне нет, достраивает программа, а правила CSS проекта накладываются сверху и меняют только то, что в них задано явно. Колонтитулы, титул и размер листа из шаблона не переносятся. Если шаблон не открылся, в журнале сборки появится предупреждение, а документ соберётся без него.

Стили таблиц шаблона тоже работают: таблица, класс которой совпадает с названием стиля таблицы в шаблоне, получает этот стиль, остальные — стиль, выбранный в списке «Стиль таблиц». Рамки и заливку такой таблицы даёт стиль Word; CSS для неё главнее, только если задан явно.

Оглавление, таблицы, картинки и колонтитулы из CSS

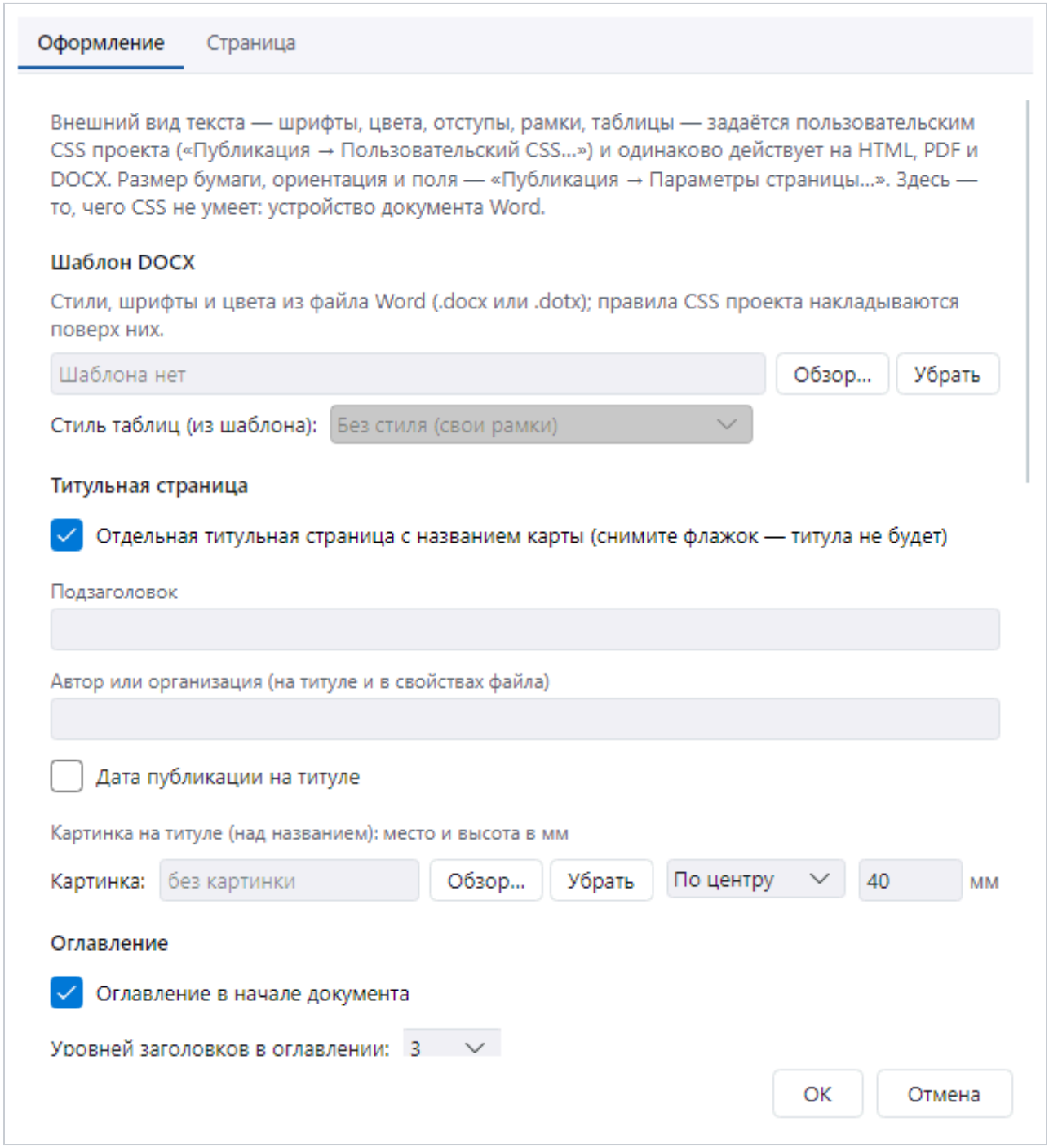
Оглавление оформляется классами `.toc`, `.toc-title`, `.toc .lvl1`…`.lvl6` и `.toc .lvl1 > .num`: шрифт, регистр (`text-transform: uppercase`), отступы, «таб» между номером и текстом, своё название через `content`; `display: none` скрывает уровень, название или всё оглавление. Правила CSS главнее диалога «Оформление DOCX».

Таблица со своим классом оформляется отдельно: `table.blue-frame { border: 2px solid #1E90FF }` — внешние рамки, `.blue-frame th` и `.blue-frame td` — заливка и рамки ячеек. У картинок работают `width`, `height`, `max-width` и `border`, а у полей колонтитулов — высота области, интервалы, регистр и подчёркивание. Полный справочник — файл `docs/CSS_REFERENCE.md`.

Настройки «Оформление DOCX»

Диалог Публикация → Оформление DOCX… задаёт то, чего нет в CSS. Настройки сохраняются вместе с проектом (файл `.ditastudio-docx`).

Рисунок 25. Окно «Оформление DOCX», вкладка «Оформление»



- титульная страница: название карты, подзаголовок, автор, дата;
- оглавление, число уровней в нём и его название («Содержание», «Оглавление» или своё) — оно сразу заполнено строками-ссылками на топики, а номера страниц Word проставит при открытии документа;
- нумерация заголовков «1», «1.1», «1.1.1» — номера видны и в оглавлении, а также в едином HTML, PDF и на сайте; абзац, отмеченный кнопкой № (нумерованный абзац), получает номер следующего уровня — во 2-й главе, 3-м разделе 2.3.1;
- топик верхнего уровня с новой страницы, нумерация рисунков и таблиц;

- колонтитулы с полями `{page}`, `{pages}`, `{title}`, `{date}` и выравниванием, без колонтитулов на титуле;
- картинка в колонтитуле (например, логотип) — кнопка **Обзор...**, место слева, по центру или справа и высота в миллиметрах; та же картинка выводится и в колонтитулах PDF;
- зеркальные поля и переплёт для двусторонней печати;
- язык текста для проверки правописания и автоматические переносы.

3.8 Переименование и перенос без битых ссылок

Как переименовать `id`, перенести файл или вынести повторяющийся фрагмент в `conref` так, чтобы все ссылки в проекте обновились сами.

3.8.1 Контекст

Ссылки `href` и `conref` указывают на путь к файлу и `id` элемента. Если переименовать их вручную, ссылки в других файлах сломаются. Команды рефакторинга находят и исправляют такие ссылки по всему проекту. Ссылки через ключи (`keyref`, `conkeyref`) от пути и `id` не зависят и не меняются.

Порядок действий

1. **Чтобы переименовать `id`, поставьте курсор в элемент и выберите Структура → Переименовать id....**

   Если у элемента нет `id`, сначала задайте его на вкладке «Атрибуты».

   → Строка состояния показывает, сколько ссылок обновлено.

2. **Чтобы переименовать или перенести файл, щёлкните его правой кнопкой на вкладке «Проект» и выберите Переименовать / переместить файл....**

   Папка назначения создаётся, если её нет. Относительные ссылки внутри самого файла пересчитываются от нового расположения, а ссылки на него из других файлов обновляются.

3. **Чтобы использовать фрагмент повторно, поставьте курсор в элемент и выберите Структура → Вынести в conref....**

   В диалоге задайте `id` вынесенного элемента и выберите существующий топик или новый файл (по умолчанию — `reusable/shared.dita`).

   → Содержимое элемента переносится в целевой топик, а на исходном месте остаётся пустой элемент со ссылкой `conref`. Текст дальше правится в целевом топике.

4. **Проверьте изменённые документы и сохраните их (Ctrl+Shift+S).**

   Перенесённый файл и новый файл для `conref` записываются на диск сразу, остальные изменения — только после сохранения.

3.8.2 Результат

Чтобы убедиться, что битых ссылок не осталось, запустите проверку проекта.

3.9 Рецензирование и сравнение версий

Как отметить изменённые места, вести правки с возможностью принять или отклонить их и сравнить документ с другой версией.

Порядок действий

1. **Чтобы отметить изменённый фрагмент для читателя, поставьте курсор в элемент и выберите Структура → Отметить как изменённое (rev).**

   → Элемент получает атрибут `rev="changed"`, в публикации у него появляется полоса на полях. Повторный выбор команды снимает пометку.

2. **Чтобы вести правки для рецензента, используйте пункты Структура → Track changes: ....**

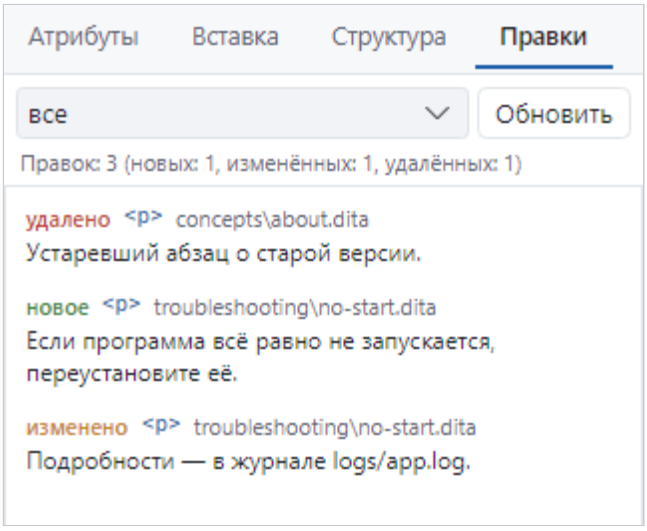
Команда	Что происходит
Пометить как вставленное	Элемент получает `status="new"`, подсвечивается в предпросмотре и остаётся в публикации.
Пометить как удалённое	Элемент получает `status="deleted"`, остаётся в документе зачёркнутым, но из публикации исключается.
Принять правку	Вставка остаётся без пометки; удалённый элемент убирается из документа.
Отклонить правку	Вставленный элемент убирается; удалённый восстанавливается.

   Вместе с пометкой записываются имя пользователя в системе и дата (атрибуты `tcauthor` и `tcdate`).

3. **Просмотрите все правки проекта на вкладке Правки справа.**

   В списке — блоки со `status` `new`, `changed` и `deleted` по всем топикам и картам: статус, элемент, файл и начало текста; фильтр по статусу и кнопка **Обновить**. Щелчок по строке открывает топик и ставит курсор в блок. Список собирается при открытии вкладки и после сохранения документа. Пометки над блоками в «Авторе» того же цвета: `new` — зелёная, `changed` — оранжевая, `deleted` — красная.

Рисунок 26. Вкладка «Правки»



4. **Чтобы сравнить два файла, выберите Правка → Сравнить файлы... и укажите оба файла.**

   → Различия показываются построчно, как в обычном diff.

5. **Чтобы перенести группу различий в другой файл, нажмите у неё кнопку → или ←.**

   > **Внимание:** Файл, в который перенесена группа, сразу записывается на диск.

6. **Чтобы сравнить открытый документ с версией из системы контроля версий, выберите Правка → Сравнить с последним коммитом (git) или Правка → Сравнить с версией BASE (svn).**

   Нужен установленный `git` или `svn` в `PATH`, файл должен быть под версионным контролем. Слева показывается версия из истории, справа — файл на диске.

3.9.1 Результат

> **Примечание:** Сравнение читает файлы с диска — сохраните документ перед сравнением, иначе несохранённые правки в нём не будут видны.

3.10 Перевод топика через XLIFF

Как выгрузить текст топика на перевод в формате XLIFF 1.2 и подставить готовый перевод обратно.

3.10.1 Контекст

XLIFF — стандартный формат обмена с переводчиками и CAT-инструментами. Каждый блочный элемент с текстом (заголовок, краткое описание, абзац, пункт списка) становится отдельным сегментом. Фразовая разметка внутри сегмента передаётся тегами: парные

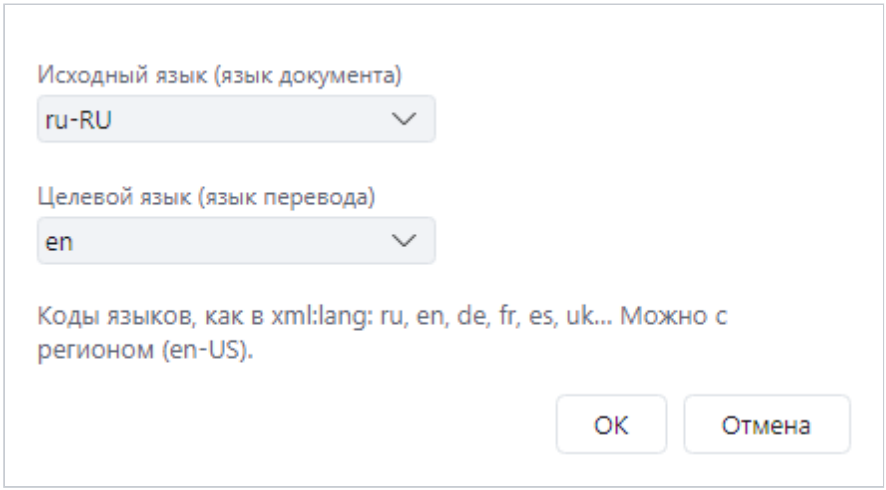
элементы — `bpt`/`ept` (текст внутри них тоже переводится), пустые (изображение, ссылка без текста) — `ph`.

Порядок действий

1. **Откройте топик и выберите Публикация → Экспорт в XLIFF (для перевода)....**

   Окно «Языки XLIFF» предлагает исходный язык (`xml:lang` документа) и целевой; можно ввести любые коды языков.

Рисунок 27. Окно «Языки XLIFF»



   → Создаётся файл `.xliff` с выбранной языковой парой.

2. **Передайте файл переводчику; перевод записывается в элементы `target`.**

3. **Откройте тот же топик и выберите Публикация → Импортировать перевод из XLIFF....**

   → Текст из `target` подставляется на место исходного; строка состояния показывает, сколько сегментов применено.

4. **Проверьте документ и сохраните его под нужным именем.**

3.10.2 Результат

Чтобы перевести сразу всю карту, выберите на вкладке «Карта» нужную карту и команду Публикация → Экспорт карты в XLIFF (все топики)...: в выбранной папке появится по файлу на топик и `manifest.json`. Готовые переводы подставляет команда Публикация → Импорт переводов карты из XLIFF...: открытые топики помечаются несохранёнными, закрытые сохраняются сразу.

> **Важно:** Сегменты сопоставляются с документом по порядку, поэтому не меняйте структуру топика между экспортом и импортом. Несовпавшие сегменты пропускаются и перечисляются в предупреждениях, остальной перевод всё равно применяется.

4 Расширение редактора

4.1 Расширение редактора

Как подключить общие ключи из другого проекта, собственные специализации DITA и плагины.

4.1.1 Проекты-источники ключей

Если общие ключи (название продукта, версия, повторяющиеся фрагменты) хранятся в отдельном проекте, подключите его через Файл → Подключить проект-источник ключей…. Ключ, которого нет в своём проекте, ищется в подключённых. Карты проекта-источника вместе с текущим проектом не публикуются, а его ключи не показываются на вкладке «Ключи». Связь сохраняется вместе с проектом.

4.1.2 Собственные специализации

Свои элементы подключаются готовым DTD: Файл → Подключить внешний DTD (кастомная специализация)…. Редактор разбирает объявления `ENTITY`, `ELEMENT`, `ATTLIST`, параметрические сущности и подключаемые модули. Новые элементы появляются в палитре вставки, автодополнении, проверке и публикации; способ их отображения выводится из атрибута `class`.

> **Примечание:** Элементы DTD действуют только в своём проекте: у каждого проекта свой каталог, так что проекты с конфликтующими специализациями не мешают друг другу. После правки файла DTD обновите проект — он перечитается.

4.1.3 Плагины

Плагины — это файлы `.dll` в папке `plugins` рядом с исполняемым файлом редактора. Они загружаются при запуске редактора и бывают трёх видов:

- правила проверки — дополнительные замечания при проверке проекта;
- форматы публикации — Публикация → Публикация плагином…;
- команды режима «Автор» — Структура → Команда плагина….

Ошибка внутри плагина не закрывает редактор — она показывается как предупреждение.

5 Справочные материалы

5.1 Главное окно

Области главного окна, их вкладки и назначение. Номера в таблице соответствуют рамкам на снимке.

Рисунок 28. Главное окно с открытым топиком

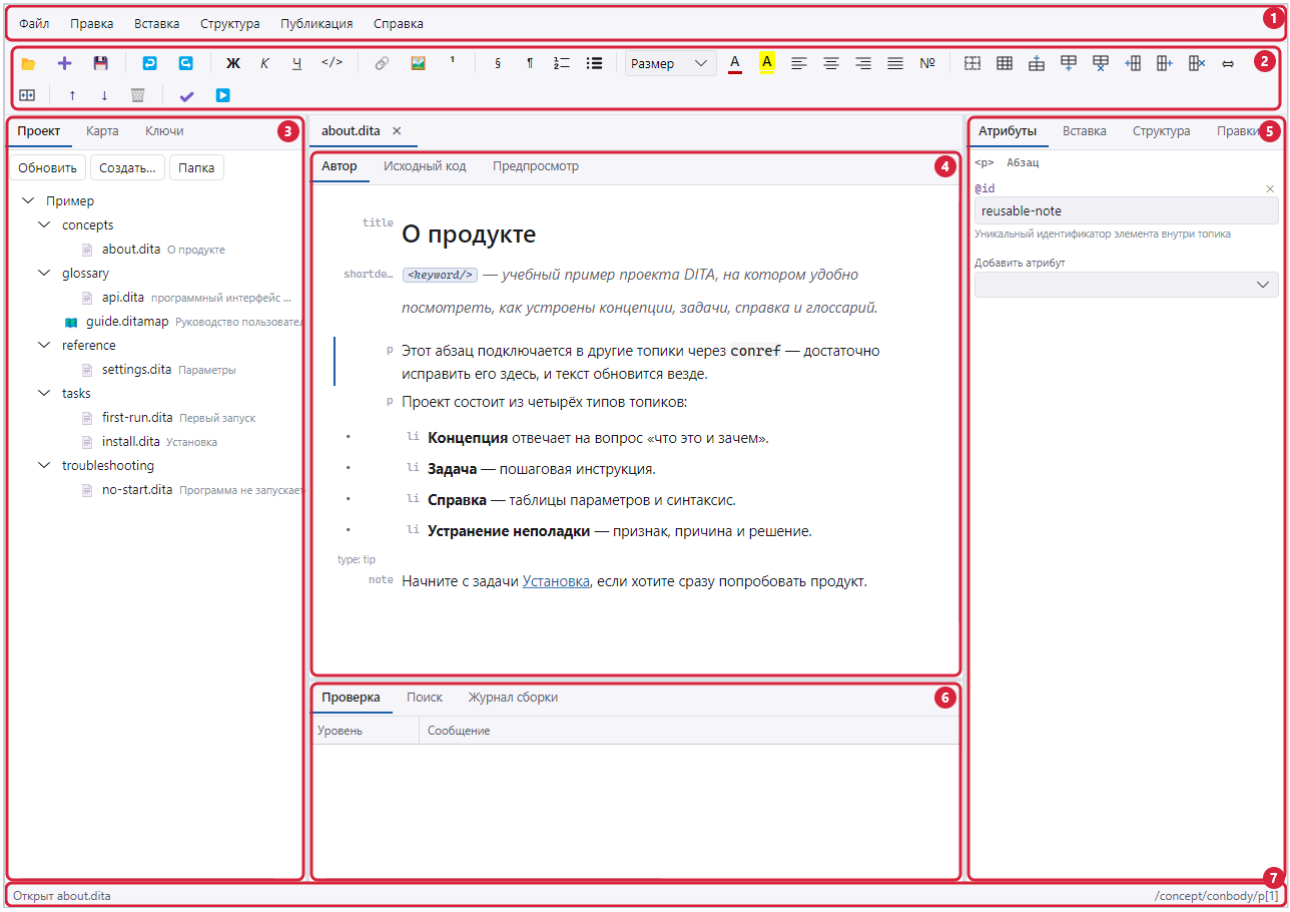


Таблица 1. Области окна

№	Область	Назначение
1	Меню	Файл, Правка, Вставка, Структура, Публикация, Справка. Пункты, недоступные в текущей позиции курсора, выключены.
2	Панель инструментов	Проект и файлы; отмена структурных правок; оформление-переключатели (`b`, `i`, `u`, `codeph`); ссылка, изображение, сноска; раздел, абзац, списки; размер, цвет, маркер, выравнивание, нумерованный абзац; таблица — строки, столбцы, объединение, границы; перемещение и удаление элемента; проверка и сборка. В узком окне переносится на вторую строку.
3	Проект, Карта, Ключи	Дерево файлов (несколько проектов — несколько корней), структура выбранной карты, ключи корневой области.
4	Документы	Вкладки открытых файлов (закрепление, «Закрыть остальные» — правой кнопкой) и режимы документа: Автор, Исходный код, Предпросмотр (HTML, PDF, DOCX приближённо, живые страницы).
5	Атрибуты, Вставка, Структура, Правки	Атрибуты элемента под курсором, палитра допустимых элементов (Ctrl+E), дерево топика, блоки со `status` по проекту.
6	Проверка, Поиск, Журнал сборки	Результаты F7 с переходом к месту, поиск и замена по проекту (Ctrl+Shift+F), вывод последней публикации.
7	Строка состояния	Итог последней операции или причина отказа; справа — путь к элементу под курсором (`/concept/conbody/p[1]`).

5.2 Сочетания клавиш

Все горячие клавиши, зарегистрированные в главном окне редактора.

Таблица 2. Файл и правка

Сочетание	Действие
Ctrl+Shift+O	Открыть папку проекта
Ctrl+N	Создать документ
Ctrl+S	Сохранить документ
Ctrl+Shift+S	Сохранить всё
Ctrl+W	Закрыть вкладку
Ctrl+Alt+Z	Отменить структурное изменение
Ctrl+Alt+Y	Повторить структурное изменение
Ctrl+Shift+F	Найти в проекте
F7	Проверить проект
F1	Справка по элементу под курсором

Таблица 3. Структура и форматирование

Сочетание	Действие
Ctrl+E	Фокус на палитре вставки
Ctrl+Space	Автодополнение элементов и атрибутов в режиме «Исходный код»
Ctrl+Shift+↑	Переместить элемент вверх
Ctrl+Shift+↓	Переместить элемент вниз
Ctrl+Alt+→	Объединить ячейку таблицы с соседней справа
Ctrl+Alt+↓	Объединить ячейку таблицы с ячейкой снизу
Ctrl+B	Полужирный
Ctrl+I	Курсив
Ctrl+U	Подчёркнутый
Ctrl+`	Код (`codeph`)
Ctrl+Shift+U	Элемент интерфейса (`uicontrol`)
Ctrl+Shift+Space	Снять оформление

Таблица 4. Публикация

Сочетание	Действие
F5	Собрать сайт HTML

5.3 Структура топиков

Сводные таблицы: общие части всех топиков, корневые элементы и тела типов, части задачи и шага.

5.3.1 Общие части

Часть	Элемент	Назначение
Заголовок	`title`	Обязателен. Заголовок страницы и строка оглавления.
Краткое описание	`shortdesc`, `abstract`	Суть топика в одном-двух предложениях. Показывается в подсказках ссылок и в поиске.
Пролог	`prolog`	Метаданные: автор, даты, ключевые слова. В текст публикации не выводится.
Тело	зависит от типа	Основное содержимое.
Связанные ссылки	`related-links`	Ссылки «Смотрите также», заданные в самом топике.

5.3.2 Типы топиков

Тип	Корень / тело	Для чего	Заготовка
Концепция	`concept` / `conbody`	Что это, как устроено, зачем	есть
Задача	`task` / `taskbody`	Пошаговая инструкция	есть
Справка	`reference` / `refbody`	Параметры, коды, синтаксис	есть
Устранение неполадки	`troubleshooting` / `troublebody`	Признак, причина, решение	есть
Статья глоссария	`glossentry` / `glossBody`	Определение термина	есть
Группа глоссария	`glossgroup`	Несколько статей в одном файле	нет
Универсальный топик	`topic` / `body`	Свободная структура	есть
Учебные топики	`learningContent` и др.	Учебные курсы	нет

5.3.3 Части задачи

Все части необязательны и идут только в этом порядке.

Элемент	Содержание
`prereq`	Что должно быть готово до начала
`context`	Зачем и когда выполняется задача
`steps`, `steps-unordered`, `steps-informal`	Шаги по порядку, в любом порядке или свободным текстом
`result`	Что получится в конце
`tasktroubleshooting`	Что делать, если результат не достигнут
`example`	Пример выполнения, может быть несколько
`postreq`	Что сделать после

5.3.4 Части шага

Элемент	Содержание
`cmd`	Обязательно, первым: одно действие в повелительном наклонении
`info`	Пояснение к действию
`substeps`	Подшаги
`stepxmp`	Пример к шагу
`choices`, `choicetable`	Варианты действия списком или таблицей
`stepresult`	Что произойдёт после шага
`steptroubleshooting`	Что делать, если шаг не удался

5.4 Что редактируется только в исходном коде

Режим «Автор» показывает не каждый элемент как редактируемый текст — вот по какому признаку это определяется.

Таблица 5. Поведение элементов в режиме «Автор»

Категория	Пример	Поведение
Блочные элементы	`p`, `note`, `table`, `steps`	Каждый — собственная редактируемая область.
Фразовые элементы с текстом	`b`, `i`, `u`, `codeph`, `uicontrol`	Редактируются прямо внутри абзаца как оформленный текст.
Инлайн-элементы без своего текста	`xref` без текста, `indexterm`, фразы с `conref`/ `conkeyref`	Показываются нередактируемой «плашкой» с меткой элемента; изменить атрибуты или вложенное содержимое можно только в режиме **Исходный код**. Исключение — изображение и `xref` без текста: щелчок по ним открывает окно правки атрибутов.
Служебные блоки	`prolog`, `topicmeta`, `metadata`	Показываются свёрнутым нередактируемым XML — правка только в режиме **Исходный код**.
Изображения	`image`	Показываются картинкой; размер меняется мышью за маркер в углу (атрибут `width`), адрес файла и подпись — в режиме **Исходный код** или на панели «Атрибуты».
Сноски	`fn`	В тексте — плашка «сноска N»; текст сноски правится в области «Сноски» внизу топика.

6 Устранение неполадок

6.1 PDF не экспортируется

Команда «Экспорт в PDF» завершается ошибкой или не создаёт файл.

6.1.1 Признак

После выбора Публикация → Экспорт в PDF файл не появляется в папке вывода, либо редактор сообщает, что не может найти браузер.

Причина

Собственного движка PDF в редакторе нет: собранный HTML печатается в файл браузером на движке Chromium — встроенным в редактор или установленным Microsoft Edge, Google Chrome, Chromium. Если встроенный браузер не запустился (см. «Вкладка „Предпросмотр“ пустая»), а установленного нет, печатать нечем.

Установить браузер или восстановить встроенный

Порядок действий

1. **Установите Microsoft Edge, Google Chrome или Chromium.**
2. **Повторите Публикация → Экспорт в PDF.**

   → Файл PDF появляется в выбранной папке вывода.

Причина

Прежний PDF с тем же именем открыт в программе просмотра, и заменить его нельзя.

Решение

Порядок действий

1. **Закройте PDF в программе просмотра и повторите экспорт.**

6.2 Вкладка «Предпросмотр» пустая

Вместо готовой страницы вкладка «Предпросмотр» показывает пустое окно или сообщение о том, что встроенный браузер недоступен.

6.2.1 Признак

При переключении на вкладку **Предпросмотр** область остаётся пустой, либо появляется сообщение «Встроенный браузер недоступен» с кнопкой открыть файл внешним приложением.

Причина

Предпросмотр рисует встроенный браузер (Chromium), который поставляется вместе с редактором. Он не запускается, если в системе нет нужных ему библиотек — чаще всего это минимальные сборки Linux без графического окружения — или если папка `CefGlueBrowserProcess` рядом с редактором повреждена.

Открыть предпросмотр снаружи или восстановить браузер

Порядок действий

1. **Нажмите кнопку в сообщении или Открыть внешним приложением: тот же предпросмотр откроется в браузере по умолчанию.**
2. **В Linux установите библиотеки рабочего стола, которые нужны Chromium (`libnss3`, `libatk1.0`, `libgbm1`, `libxkbcommon0`).**
3. **Если сообщение осталось, распакуйте архив редактора заново и перезапустите DITA Studio.**

→ Топик или карта отображаются так же, как после публикации в HTML.

7 Глоссарий

7.1 топик

Минимальная самостоятельная единица содержимого DITA — один XML-файл определённого типа (концепция, задача, справка, устранение неполадки, статья глоссария), который можно переиспользовать в разных картах.

7.2 карта

Файл `.ditamap`, который собирает топики в структуру издания: задаёт порядок разделов, вложенность, ключи (`keydef`) и заголовки разделов, не привязанные ни к одному топику.

карта (ditamap)

7.3 ключ

Имя, объявленное один раз в `keydef` карты и связанное с текстом или файлом. Топики ссылаются на него через атрибут `keyref` — так один и тот же текст (например, название продукта) можно исправить в одном месте и обновить сразу во всех топиках.

ключ (keyref)